



UNIVERSIDADE
FEDERAL DO CEARÁ

Relatório Final

Manutenção De Software - 01 - 2026.1

Profa. Carla Ilane Moreira Bezerra

Autores

Graziella Rodrigues de Lima

Wania Kelly dos Santos Oliveira

Sumário

Sumário	1
1. Caracterização do Projeto	3
2. Métricas Antes da Refatoração	3
2.1 Radon Antes da Refatoração	3
2.1.1 Complexidade Ciclomática por Arquivo Antes da Refatoração	3
2.1.2 Complexidade Ciclomática por Função Antes da Refatoração	4
2.1.3 Índice de Manutenibilidade Antes da Refatoração	4
2.1.4 Métricas brutas Antes da Refatoração	5
2.2 CodeCarbon Antes da Refatoração	5
2.3 Pylint Antes da Refatoração	5
2.3.1 Distribuição do Smells Antes da Refatoração	5
2.3.2 Distribuição dos Problemas de Código Antes da Refatoração	6
2.3.3 Arquivos mais Críticos Antes da Refatoração	6
2.3.4 Score Projeto Antes da Refatoração	7
2.4 Pytest Antes da Refatoração	7
2.4.1 Cobertura de testes Antes da Refatoração	7
2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração	9
3. Processo de Refatoração	9
3.1 too-many-statements	9
3.1.1 Ocorrência Número 1	9
3.1.2 Ocorrência Número 2	10
3.2 too-many-instance-attributes	11
3.3 too-many-branches	11
4. Métricas Após a Refatoração	11
4.1 Radon Após a Refatoração	11
4.1.1 Complexidade Ciclomática por Arquivo Após a Refatoração	11
4.1.2 Complexidade Ciclomática por Função Após a Refatoração	11
4.1.3 Índice de Manutenibilidade Após a Refatoração	12
4.1.4 Métricas Brutas Após a Refatoração	12
4.2 CodeCarbon Após a Refatoração	12
4.3 Pylint Após a Refatoração	13
4.3.1 Distribuição do Smells Após a Refatoração	13
4.3.2 Distribuição dos Problemas de Código Após a Refatoração	13
4.3.3 Arquivos mais Críticos Após a Refatoração	13
4.3.4 Score Projeto Após a Refatoração	13
4.4 Pytest Após a Refatoração	13
4.4.1 Cobertura de testes Após a Refatoração	13
4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração	13

5. Comparação dos Resultados	14
6. Percepções sobre a Prática	14
7. Links	14

1. Caracterização do Projeto

Nome Projeto: Piccolo

O Piccolo é um projeto open source desenvolvido em Python que oferece um ORM (*Object-Relational Mapper*) é um query builder moderno, rápido e amigável ao desenvolvedor.

As principais capacidades do Piccolo incluem:

- Suporte a operações síncronas e assíncronas;
- Construção de consultas SQL por meio de uma API Python expressiva;
- Recursos integrados, como autenticação, migrações, modelo de usuário e interface administrativa;
- Suporte a anotações de tipo modernas em Python;
- Organização modular por meio de Piccolo apps;
- Integração com aplicações ASGI e frameworks como FastAPI, Starlette, Litestar, Quart, Falcon e Sanic.

O propósito do Piccolo é simplificar o desenvolvimento de aplicações Python que utilizam bancos de dados relacionais, oferecendo uma alternativa moderna a ORMs tradicionais. O projeto busca combinar produtividade, clareza de código e suporte nativo a programação assíncrona, característica importante para aplicações web modernas. Além disso, o Piccolo também fornece ferramentas complementares, como migrações, playground, autenticação e painel administrativo, reduzindo a necessidade de bibliotecas externas para tarefas comuns no desenvolvimento de sistemas

2. Métricas Antes da Refatoração

2.1 Radon Antes da Refatoração

2.1.1 Complexidade Ciclomática por Arquivo Antes da Refatoração

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
piccolo/utils/pydantic.py	6	9.83	40	59	E	create_pydantic_model
piccolo/apps/migrations/auto/migration_manager.py	37	4.03	29	149	D	_run_alter_columns
piccolo/query/methods/select.py	33	3.64	29	120	D	response_handler
piccolo/apps/migrations/auto/schema_snapshot.py	2	15.50	25	29	D	get_snapshot
piccolo/apps/migrations/auto/serialisation.py	58	2.10	24	122	D	serialise_params

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
piccolo/utils/uuid.py	1	1.00	1	1	A	uuid7
piccolo/engine/finder.py	1	1.00	1	1	A	engine_finder
piccolo/apps/meta/commands/version.py	1	1.00	1	1	A	version
piccolo/apps/user/piccolo_migrations/2019-11-14T21-52-21.py	1	1.00	1	1	A	forwards
piccolo/apps/user/piccolo_migrations/2021-04-30T16-14-15.py	1	1.00	1	1	A	forwards

2.1.2 Complexidade Ciclométrica por Função Antes da Refatoração

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
piccolo/utils/pydantic.py	function	create_pydantic_model	E	40
piccolo/apps/migrations/auto/migration_manager.py	method	_run_alter_columns	D	29
piccolo/query/methods/select.py	method	response_handler	D	29
piccolo/apps/migrations/auto/schema_snapshot.py	method	get_snapshot	D	25
piccolo/apps/migrations/auto/serialisation.py	function	serialise_params	D	24

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
piccolo/apps/app/commands/new.py	function	get_app_module	A	1
piccolo/apps/app/commands/new.py	function	new	A	1
piccolo/apps/asgi/commands/new.py	function	print_instruction	A	1
piccolo/apps/fixtures/commands/dump.py	function	dump	A	1
piccolo/apps/fixtures/commands/dump.py	function	dump_to_json_string	A	1

2.1.3 Índice de Manutenibilidade Antes da Refatoração

Tabela de piores métricas

arquivo	mi	rank_mi
piccolo/columns/column_types.py	0.0000	C
piccolo/apps/migrations/auto/migration_manager.py	21.3992	A
piccolo/table.py	22.8621	A
piccolo/query/mixins.py	24.6094	A
piccolo/query/methods/select.py	28.3498	A

Tabela de melhores métricas

arquivo	mi	rank_mi
piccolo/columns/defaults/interval.py	100.0000	A
piccolo/columns/__init__.py	100.0000	A
piccolo/apps/user/piccolo_migrations/2019-11-14T21-52-21.py	100.0000	A
piccolo/query/functions/__init__.py	100.0000	A
piccolo/apps/migrations/auto/operations.py	100.0000	A

2.1.4 Métricas brutas Antes da Refatoração

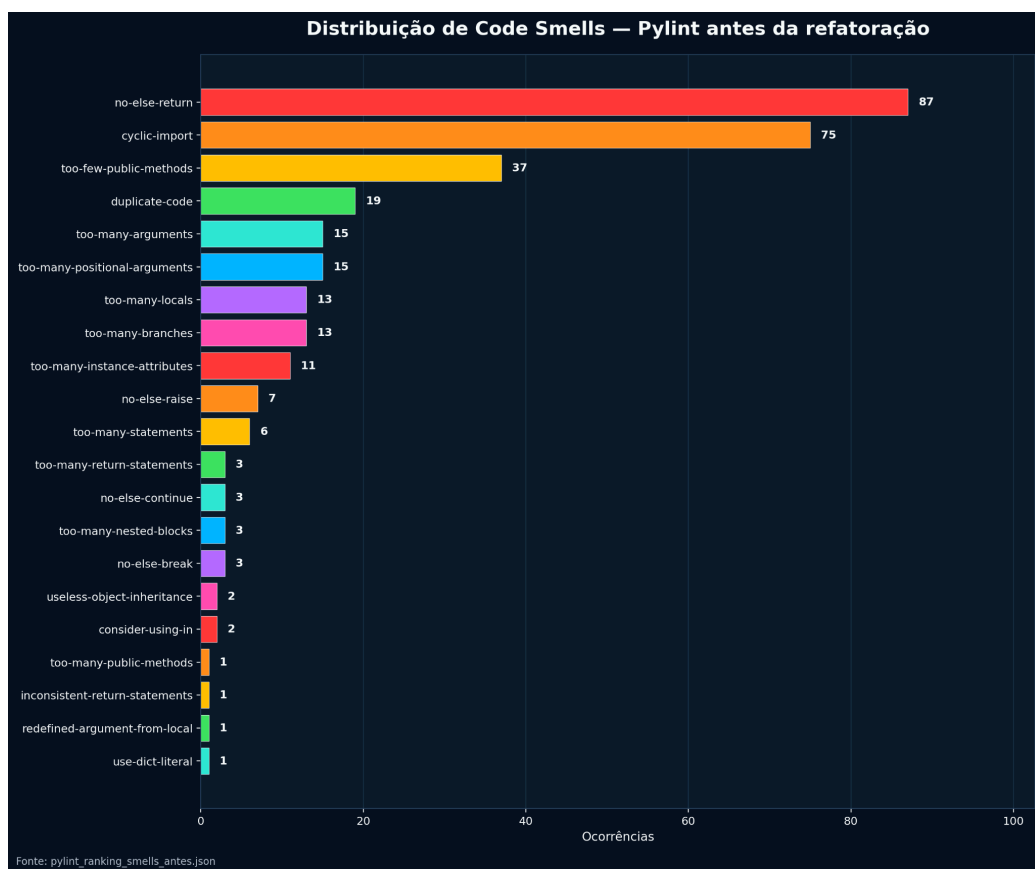
loc total	lloc total	sloc total	comments total	multi
25642	10889	15993	879	3777

2.2 CodeCarbon Antes da Refatoração

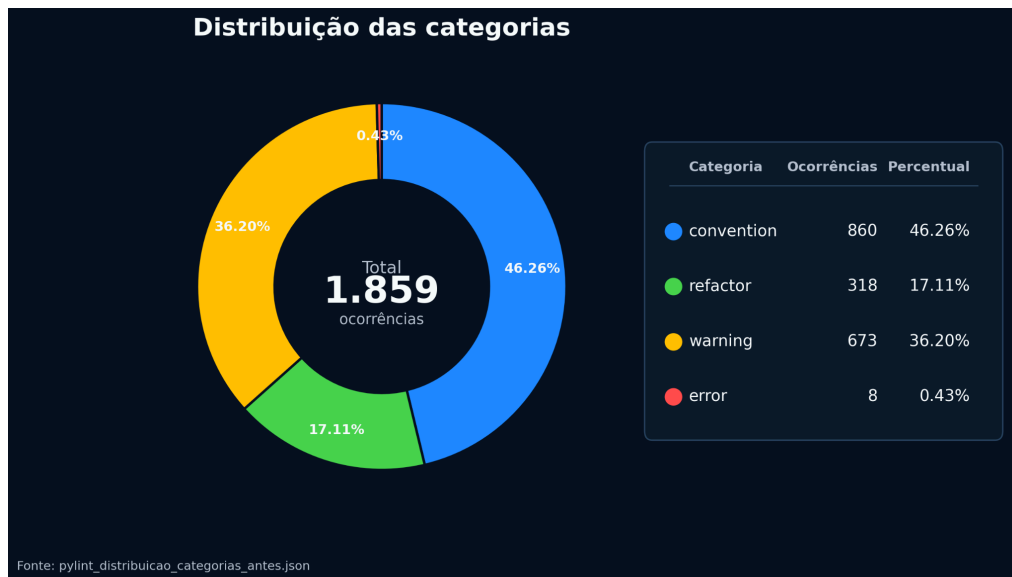
duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
2.8723951609863434	1.421226749718291e-06	4.947880323089878e-07	2.3019070783709026e-06	1.2149090905505647e-05	1.4450997983876549e-05

2.3 Pylint Antes da Refatoração

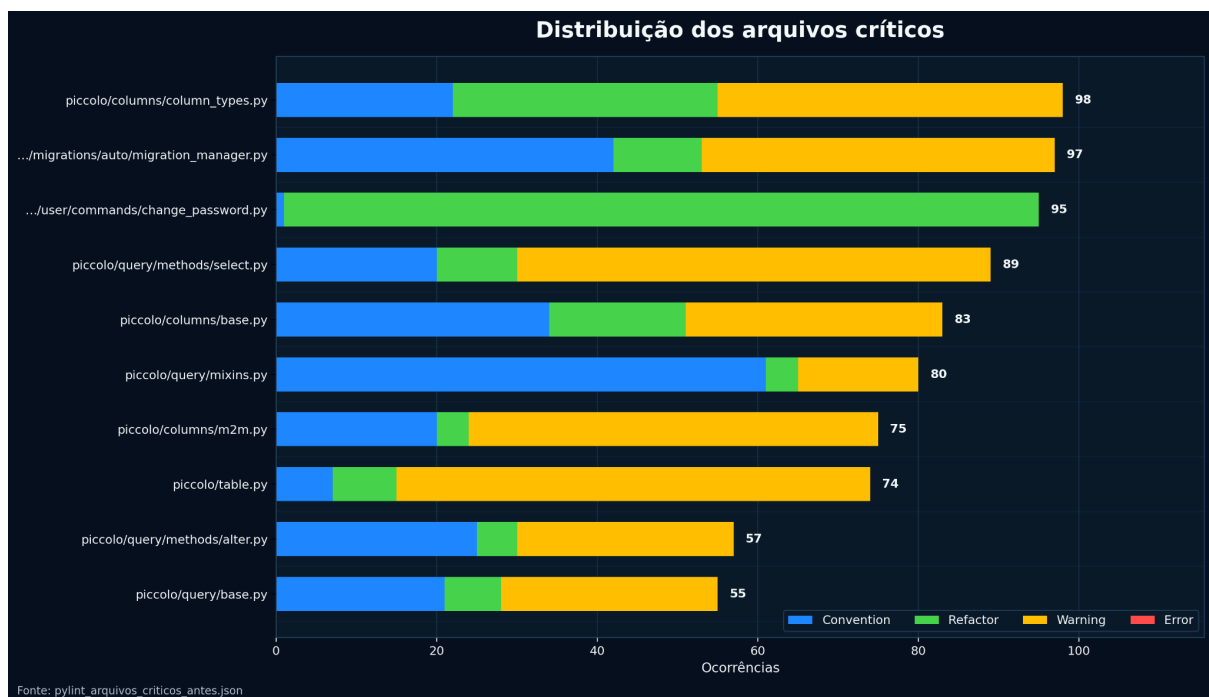
2.3.1 Distribuição do Smells Antes da Refatoração



2.3.2 Distribuição dos Problemas de Código Antes da Refatoração



2.3.3 Arquivos mais Críticos Antes da Refatoração



2.3.4 Score Projeto Antes da Refatoração

Score do projeto

7.91/10

Execução anterior: 7.91/10 | Variação: -0.00

Fonte: pylint_score_antes.txt

2.4 Pytest Antes da Refatoração

2.4.1 Cobertura de testes Antes da Refatoração

File	statements	missing	coverage
piccolo/table.py	477	24	95%
piccolo/columns/base.py	378	37	90%
piccolo/columns/column_types.py	872	111	87%
piccolo/query/methods/select.py	279	28	90%
piccolo/query/methods/alter.py	282	85	70%
piccolo/query/methods/objects.py	195	11	94%
piccolo/query/base.py	212	46	78%
piccolo/query/mixins.py	425	67	84%
piccolo/engine/sqlite.py	329	26	92%
piccolo/engine/postgres.py	255	180	29%
piccolo/engine/base.py	101	9	91%
piccolo/apps/migrations/auto/migration_manager.py	343	169	51%
piccolo/apps/migrations/auto/operations.py	52	0	100%
piccolo/apps/migrations/auto/serialisation.py	362	64	82%
piccolo/apps/migrations/auto/schema_differ.py	266	21	92%
piccolo/apps/migrations/auto/schema_snapshot.py	55	7	87%
piccolo/apps/migrations/auto/diffable_table.py	66	7	89%
piccolo/apps/migrations/commands/forwards.py	74	61	18%
piccolo/apps/migrations/commands/backwards.py	76	62	18%
piccolo/apps/schema/commands/generate.py	326	220	33%
piccolo/utils/pydantic.py	113	5	96%
piccolo/conf/apps.py	254	20	92%

2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração

Quantidade de testes que passaram: 711

Quantidade de testes que falharam: 0

3. Processo de Refatoração

Modelo utilizado: DeepSeek V4 Flash Free

3.1 too-many-statements

Quantidade de ocorrências: 6

3.1.1 Ocorrência Número 1

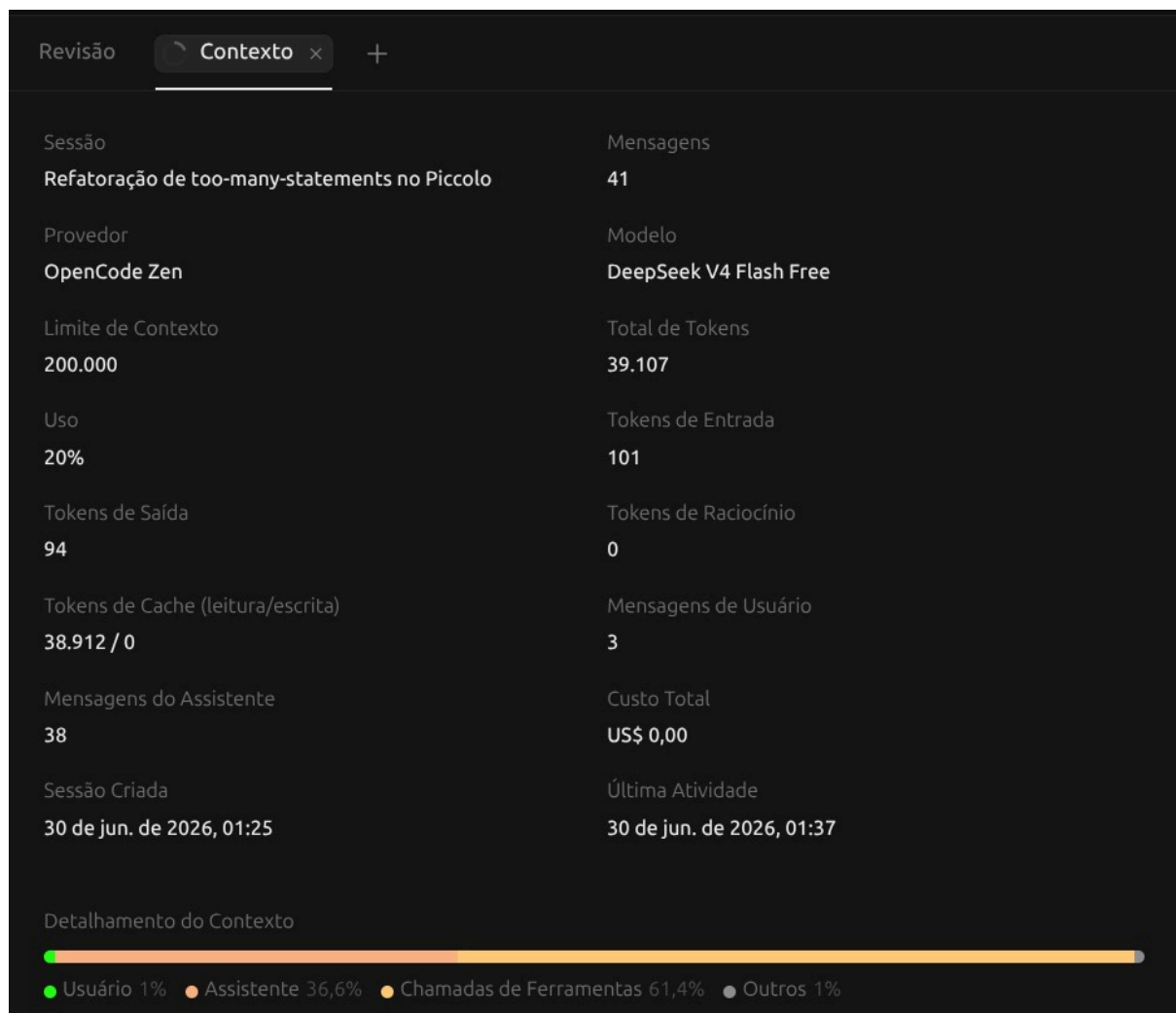
Tabela de identificação

module	obj	message
piccolo.table	Table.__init_subclass__	Too many statements (61/50)

Seguiu o plano de refatoração: Sim, mas com correção durante a implementação. O plano foi aceito porque reduzia o excesso de statements sem alterar o comportamento esperado. Porém, durante a aplicação, alguns métodos de TableMeta foram movidos por engano para _ColumnCollection. Foi necessário corrigir isso, devolvendo esses métodos para TableMeta e deixando _ColumnCollection apenas como estrutura de dados.

Link da publicação OpenCode: <https://opncd.ai/share/WX2z194u>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois corrigiu o smell R0915: too-many-statements, reduzindo `__init_subclass__` de 61 statements para cerca de 22, abaixo do limite de 50. A sessão durou cerca de 12 minutos, de 01:25 a 01:37, e consumiu 39.107 tokens, aproximadamente 20% do contexto. O custo foi moderado, e houve retrabalho por causa do erro estrutural inicial, mas a solução final ficou adequada: o Pylint deixou de apontar R0915 e a organização do método melhorou sem mudar a API pública.

3.1.2 Ocorrência Número 2

Tabela de identificação

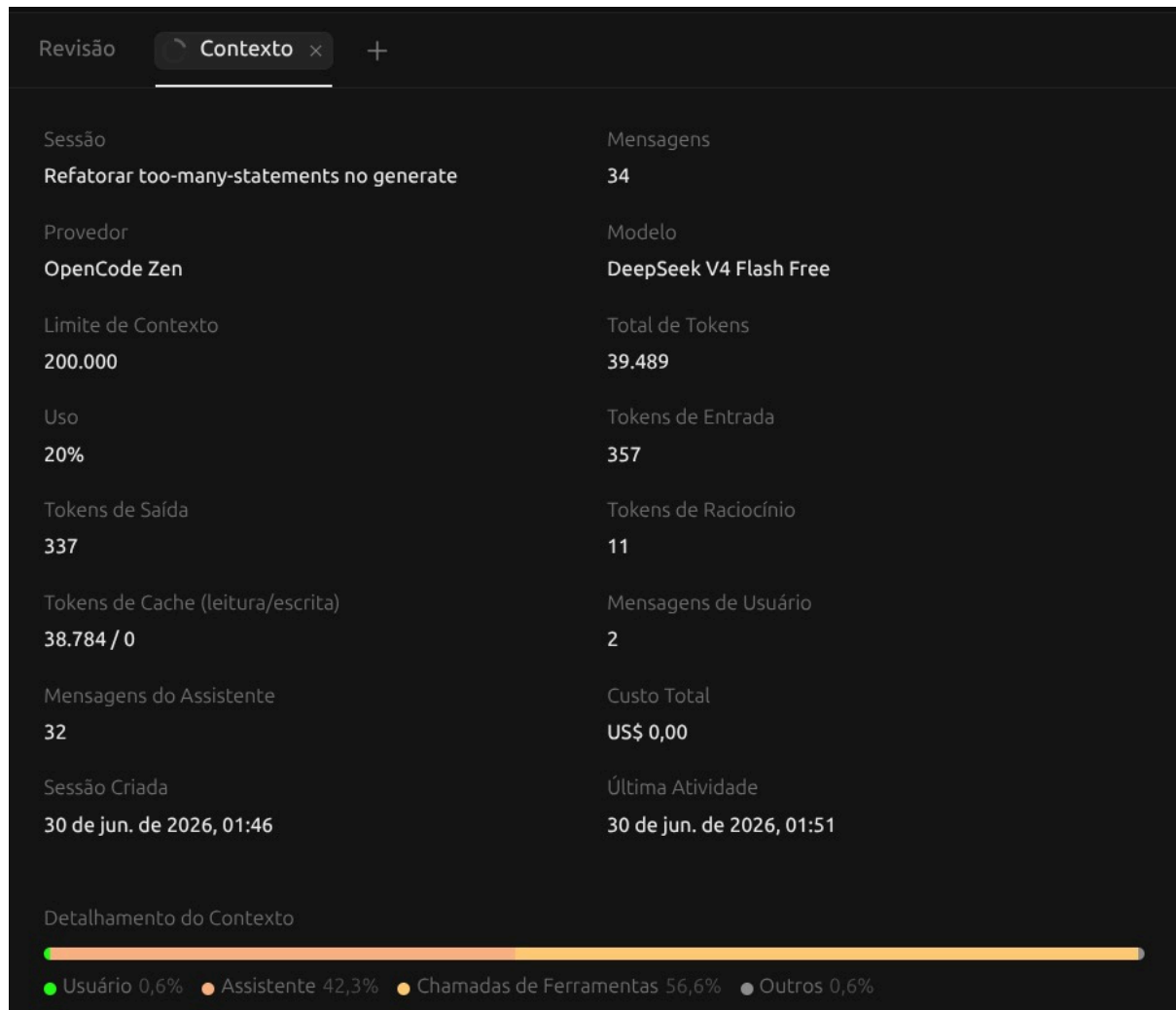
module	obj	message
piccolo.apps.schema.commands.generate	create_table_class_from_db	Too many statements (72/50)

Seguiu o plano de refatoração: Sim, com pequenos ajustes técnicos. O plano foi aceito porque separava responsabilidades sem alterar a lógica de negócio. Durante a implementação, foi necessário

ajustar a assinatura da função auxiliar para remover um parâmetro não utilizado e evitar novos avisos do Pylint, mantendo a refatoração limpa.

Link da publicação OpenCode: <https://opncd.ai/share/IYENDYiH>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois corrigiu o smell R0915: too-many-statements em uma função grande e complexa, reduzindo a contagem de cerca de 72 para 42 statements. A sessão durou cerca de 5 minutos e consumiu 39.489 tokens, aproximadamente 20% do contexto. O custo foi moderado, mas aceitável, já que o plano foi seguido sem retrabalho relevante e a solução melhorou a legibilidade ao isolar a lógica de FK e de configuração de tipos em helpers específicos.

3.1.3 Ocorrência Número 3

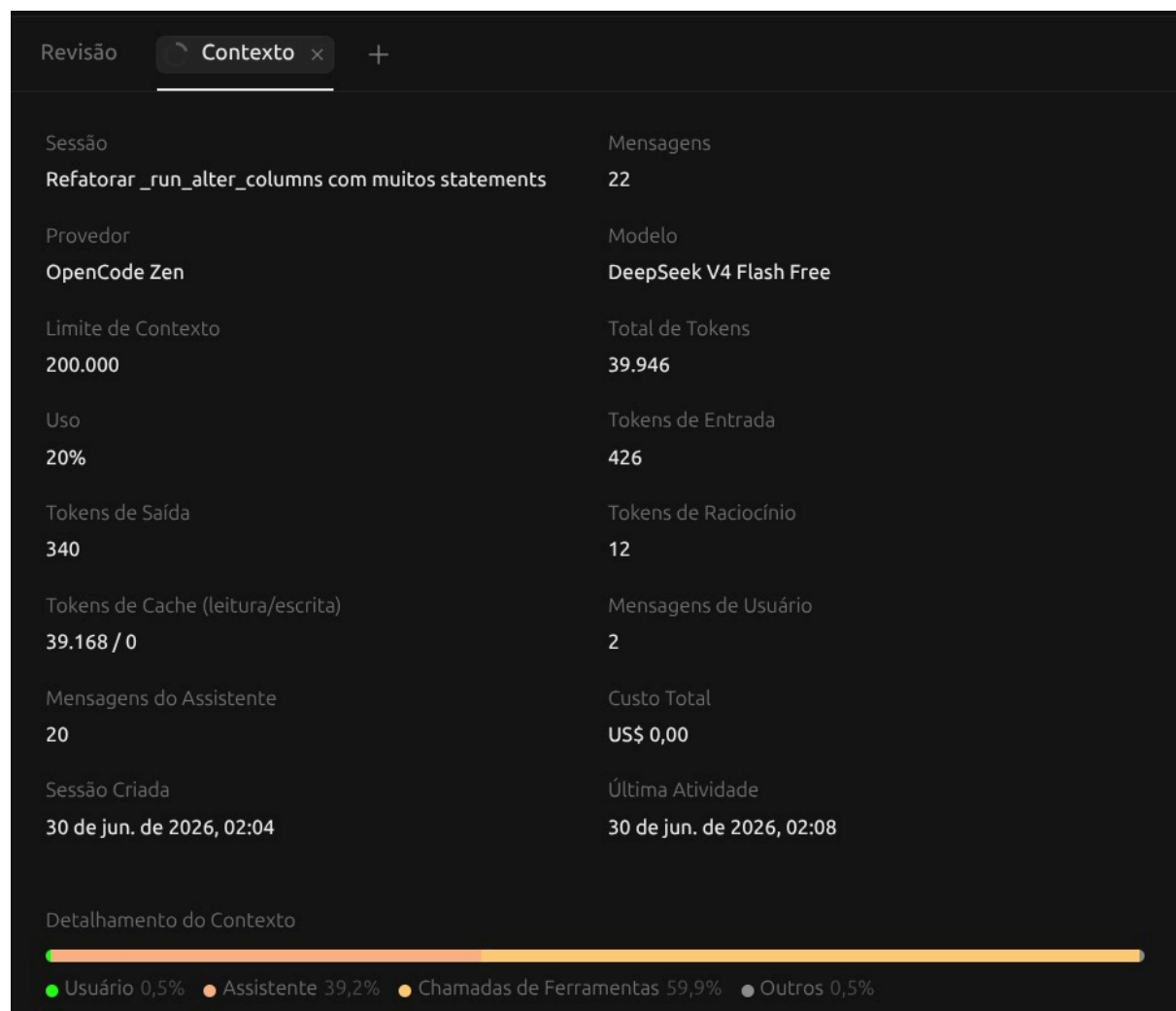
Tabela de identificação

module	obj	message
piccolo.apps.migrations.auto.migration_manager	MigrationManager._run_alter_columns	Too many statements (85/50)

Seguiu o plano de refatoração: Sim. O plano foi aceito de primeira, pois aplicava uma extração de métodos direta e coerente, reduzindo o tamanho de `_run_alter_columns` sem alterar a lógica de migração.

Link da publicação OpenCode: <https://opncd.ai/share/pBM1ohcl>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois o método tinha 85 statements para um limite de 50, indicando excesso de responsabilidades em um único bloco. A sessão durou cerca de 4 minutos e consumiu 39.946 tokens, aproximadamente 20% do contexto. O custo foi moderado, mas

aceitável, já que o plano foi seguido sem retrabalho e a solução reduziu o método para cerca de 20 statements, mantendo a lógica original e eliminando o warning R0915.

3.1.4 Ocorrência Número 4

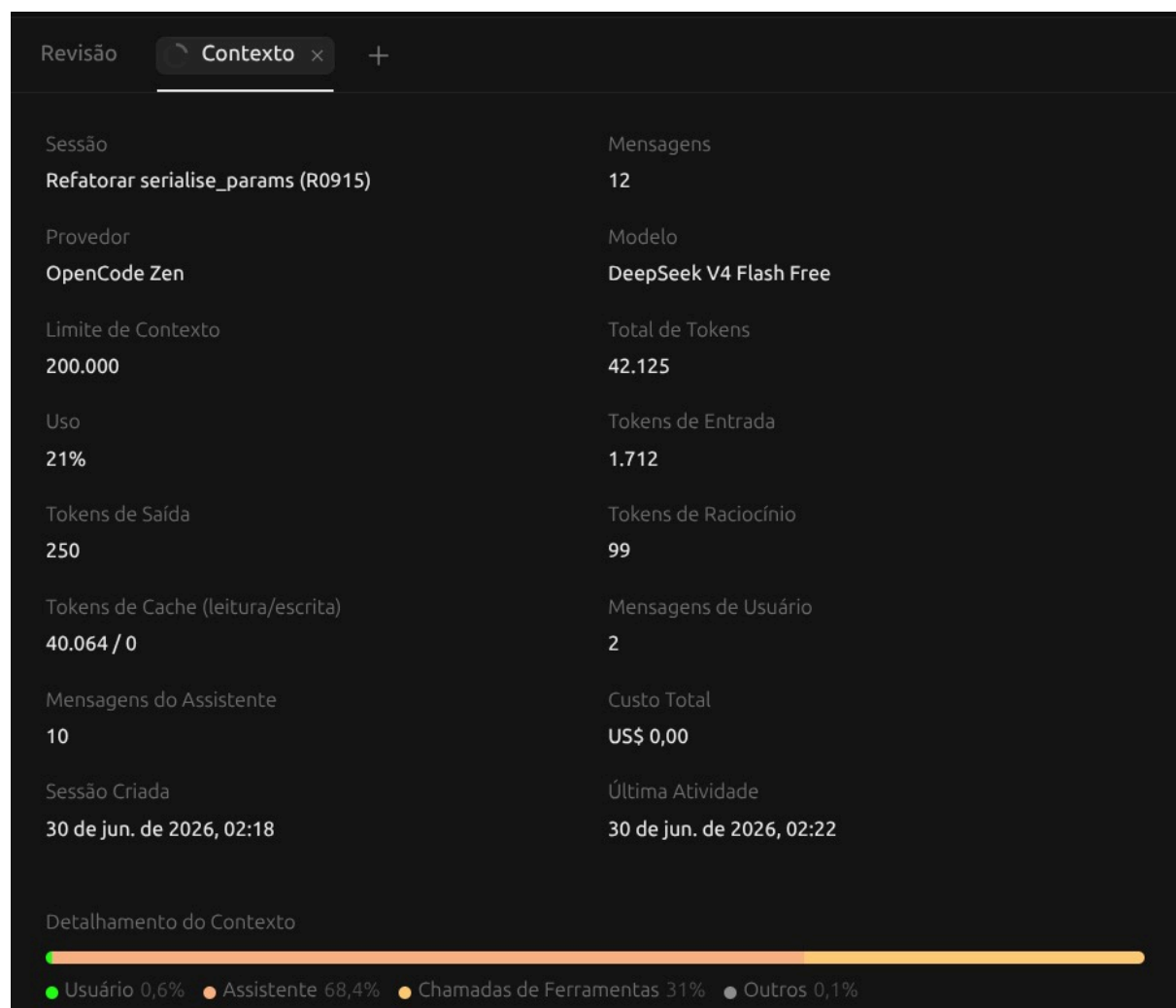
Tabela de identificação

module	obj	message
piccolo.apps.migrations.auto.serialisation	serialise_params	Too many statements (88/50)

Seguiu o plano de refatoração: Sim. O plano foi aceito de primeira, pois separava os diferentes casos de serialização em funções menores, preservando a ordem original dos branches e sem alterar a lógica de negócio.

Link da publicação OpenCode: <https://opncd.ai/share/cAl4JMCr>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois `serialize_params` tinha 88 statements para um limite de 50, tornando a função extensa e difícil de manter. A sessão durou cerca de 4 minutos e consumiu 42.125 tokens, aproximadamente 21% do contexto. O custo foi moderado, mas aceitável, já que o plano foi seguido sem retrabalho e a função principal foi reduzida para cerca de 15 instruções, com o Pylint indicando 10/10 para R0915.

3.1.5 Ocorrência Número 5

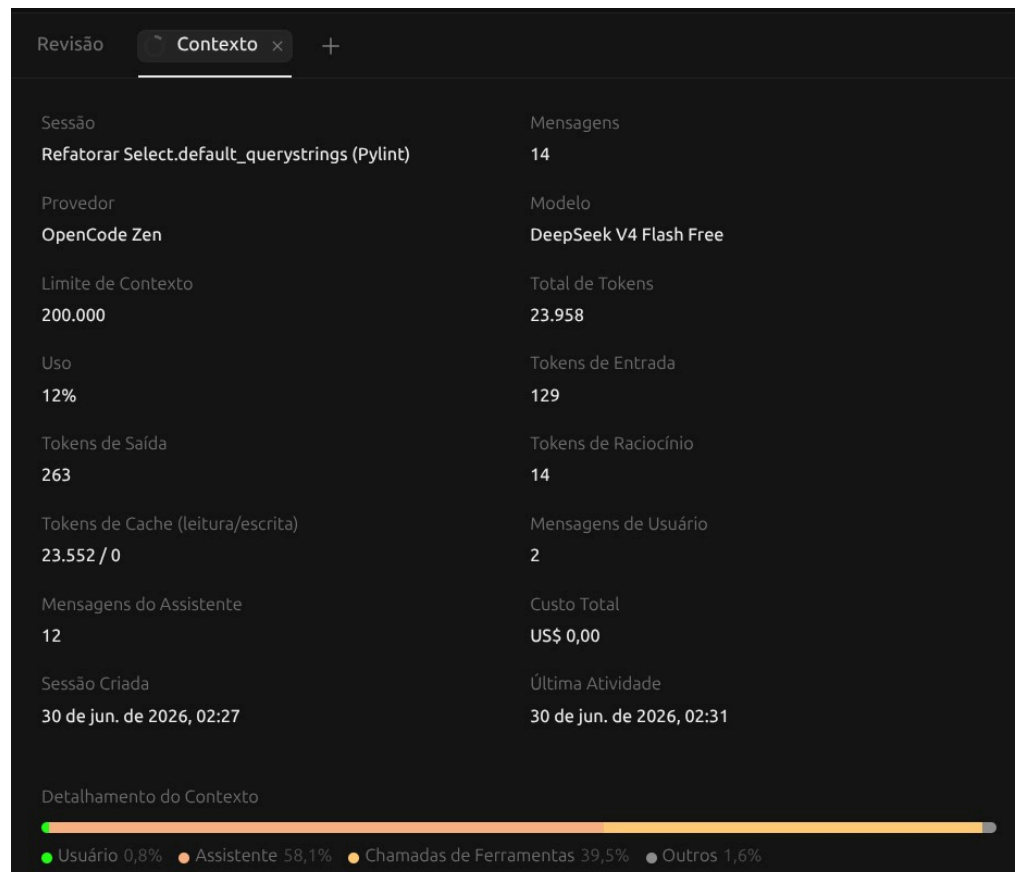
Tabela de identificação

module	obj	message
<code>piccolo.query.methods.select</code>	<code>Select.default_querystrings</code>	Too many statements (53/50)

Seguiu o plano de refatoração: Sim. O plano foi aceito de primeira, pois apenas reorganizava a lógica em métodos coesos, sem alterar a montagem da query nem a API pública.

Link da publicação OpenCode: <https://opncd.ai/share/Lvz08CWQ>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois corrigiu o smell R0915: too-many-statements com uma mudança pequena e bem delimitada. A sessão durou cerca de 4 minutos e consumiu 23.958 tokens, aproximadamente 12% do contexto. O custo foi baixo/moderado, e a solução compensou porque reduziu bastante o tamanho do método principal, mantendo a lógica original apenas distribuída em funções auxiliares.

3.1.6 Ocorrência Número 6

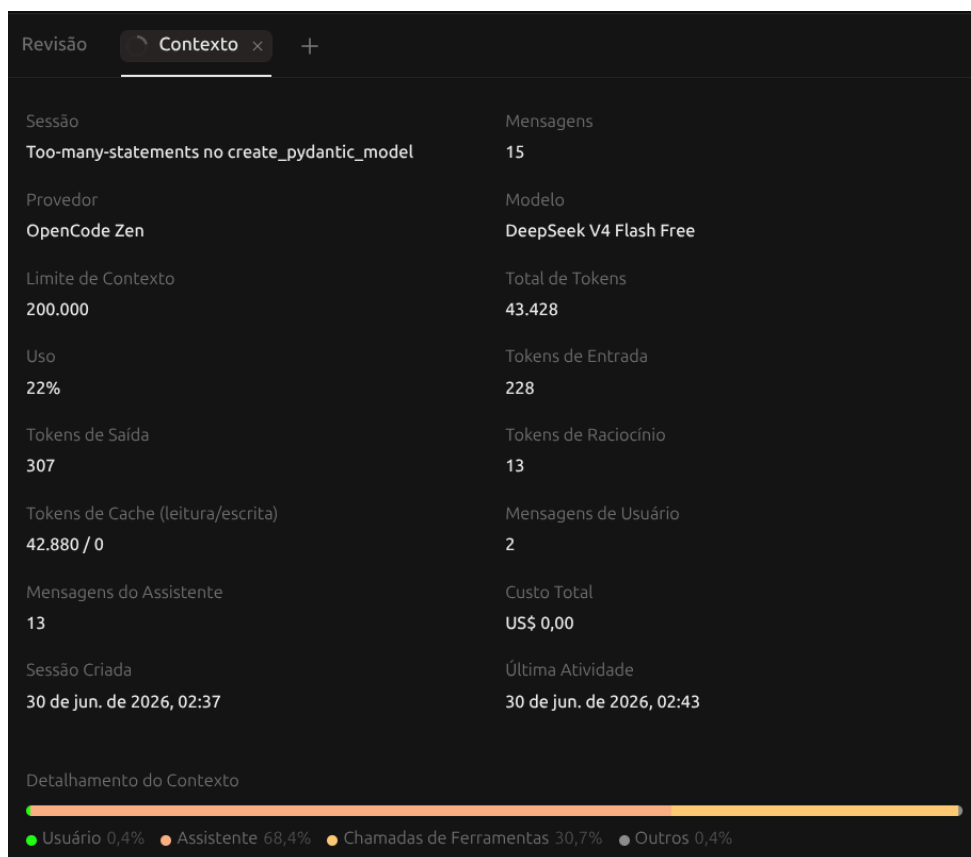
Tabela de identificação

module	obj	message
piccolo.utils.pydantic	create_pydantic_model	Too many statements (64/50)

Seguiu o plano de refatoração: Sim. O plano foi aceito de primeira, pois apenas reorganizava a lógica em métodos coesos, sem alterar a montagem da query nem a API pública.

Link da publicação OpenCode: <https://opncd.ai/share/3blmvzF7>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois corrigiu o smell R0915: too-many-statements na função `create_pydantic_model`, reduzindo a contagem de 64 para aproximadamente 20-25 statements. A sessão durou cerca de 6 minutos (02:37 às 02:43) e consumiu 43.428 tokens, aproximadamente 22% do contexto, com custo de US\$ 0,00 por utilizar o modelo DeepSeek V4 Flash Free. O plano foi seguido sem retrabalho, e a solução melhorou significativamente a legibilidade ao isolar três responsabilidades distintas (filtragem de colunas, construção de campo individual e configuração do modelo) em funções auxiliares com nomes autoexplicativos, mantendo a função principal como uma composição clara de chamadas.

3.2 too-many-instance-attributes

Quantidade de ocorrências: 11

3.2.1 Ocorrência Número 1

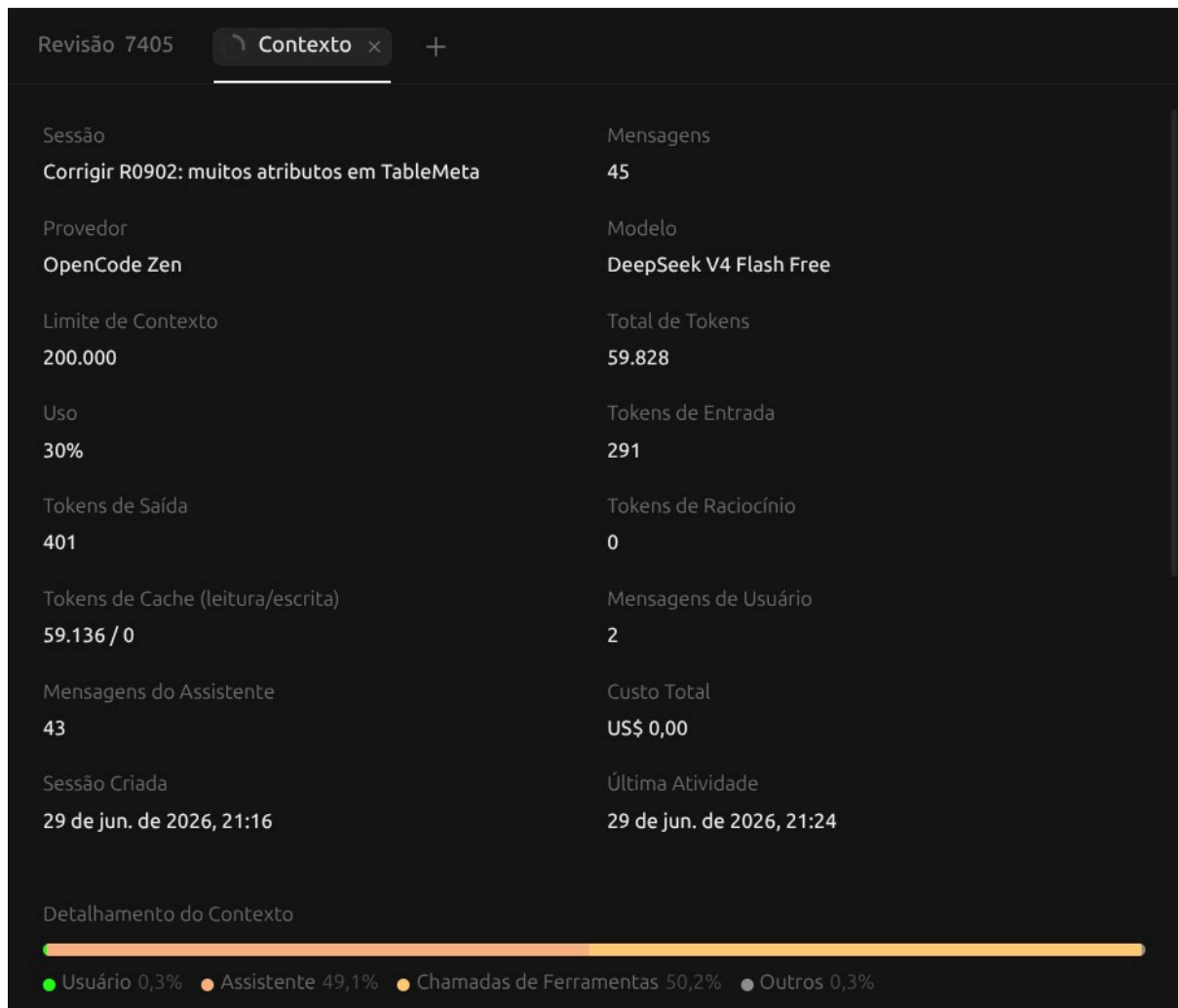
Tabela de identificação

module	obj	message
piccolo.table	TableMeta	Too many instance attributes (18/7)

Seguiu o plano de refatoração: Sim. O plano sugerido pela LLM foi seguido de primeira, pois parecia adequado para corrigir a ocorrência sem alterar o comportamento esperado do sistema.

Link da publicação OpenCode: <https://opncd.ai/share/HL3jq6Kn>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois a LLM ajudou a corrigir uma ocorrência estrutural de forma rápida. A sessão durou cerca de 8 minutos e usou 59.828 tokens, o que representa um custo moderado de contexto, mas aceitável considerando que não houve retrabalho e o plano foi aceito logo no início.

3.2.2 Ocorrência Número 2

Tabela de identificação

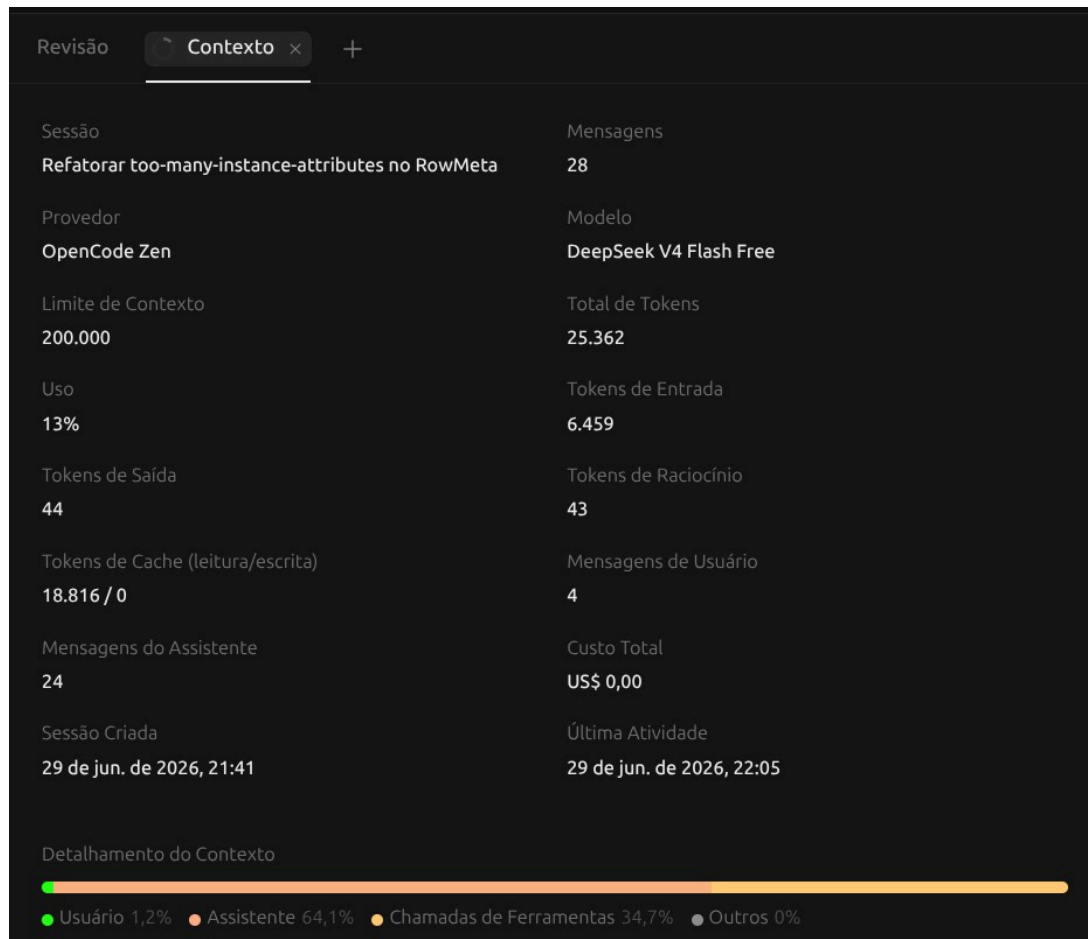
module	obj	message
piccolo.apps.schema.commands.generate	RowMeta	Too many instance attributes (9/7)

Seguiu o plano de refatoração: Não integralmente. O plano original sugeria criar um factory method `from_db_row()`, mas isso aumentaria o escopo da refatoração por exigir mudanças em pontos externos de instanciação e no uso posterior da classe. Por isso, foi solicitada uma alternativa mais

conservadora, baseada em dataclasses.InitVar, __post_init__ e properties, que corrigiu o smell sem alterar a API pública nem outros arquivos.

Link da publicação OpenCode: <https://opncd.ai/share/vwmjxu3B>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois a LLM ajudou a encontrar uma solução menos invasiva para a ocorrência too-many-instance-attributes no RowMeta. Embora o plano inicial não tenha sido seguido de primeira, a interação adicional foi útil para evitar mudanças desnecessárias no sistema. A sessão consumiu 25.362 tokens, cerca de 13% do contexto, e durou aproximadamente 24 minutos. O custo de tempo foi maior que uma simples aceitação do plano, mas compensou porque a solução final reduziu o smell preservando o comportamento esperado e limitando o impacto da refatoração.

3.2.3 Ocorrência Número 3

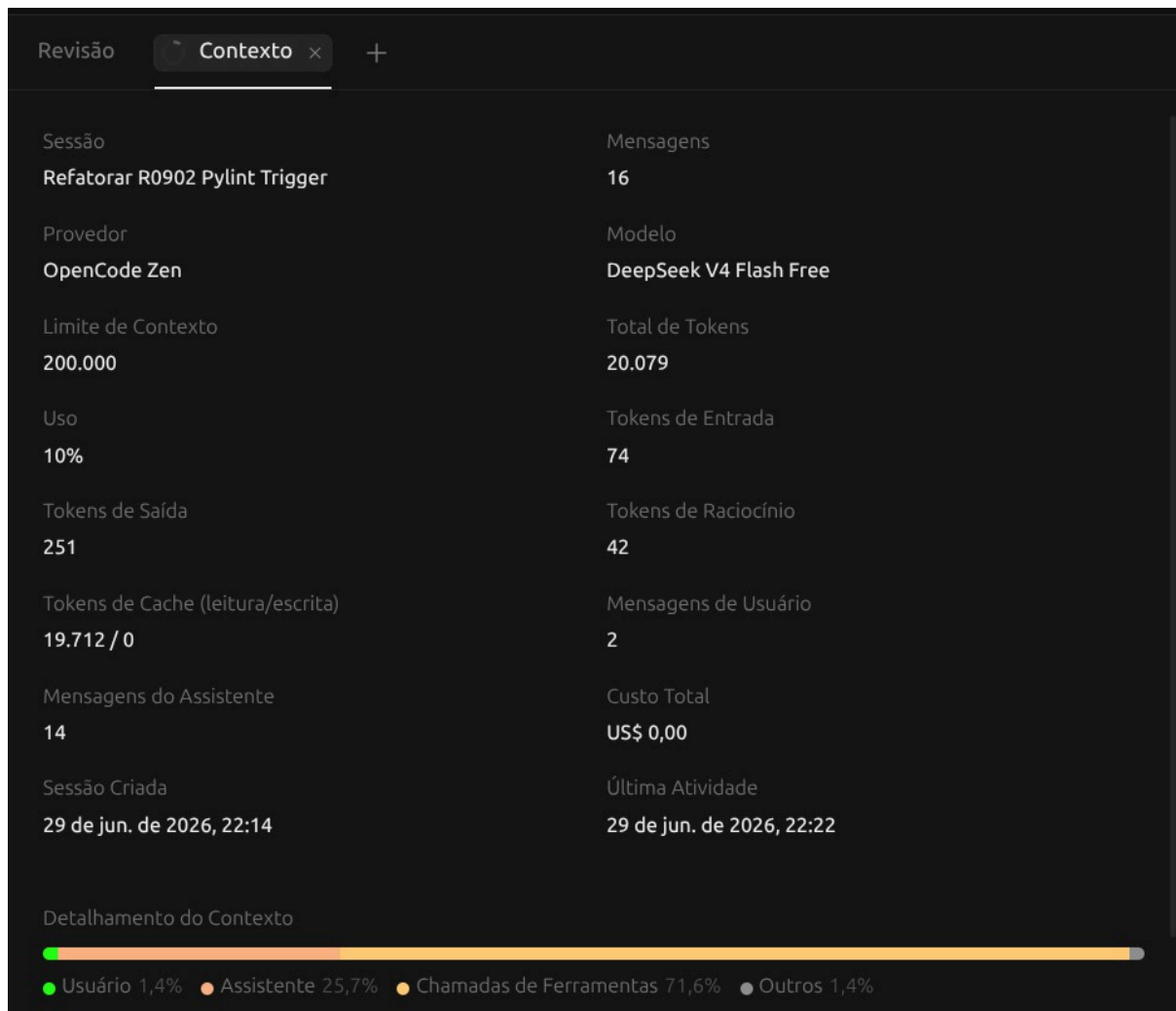
Tabela de identificação

module	obj	message
piccolo.apps.schema.commands.generate	Trigger	Too many instance attributes (8/7)

Seguiu o plano de refatoração: Sim. O plano sugerido pela LLM foi aceito de primeira, pois agrupava atributos relacionados de forma coerente, reduzia a quantidade de atributos em Trigger e mantinha a refatoração restrita ao arquivo generate.py.

Link da publicação OpenCode: <https://opncd.ai/share/JOONTLkS>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois a LLM apresentou uma solução simples e adequada para corrigir o excesso de atributos na classe Trigger. A sessão durou cerca de 8 minutos e consumiu 20.079 tokens, aproximadamente 10% do contexto disponível. Como o plano foi aceito logo na primeira tentativa, não houve retrabalho relevante, tornando o uso da LLM eficiente para essa ocorrência.

3.2.4 Ocorrência Número 4

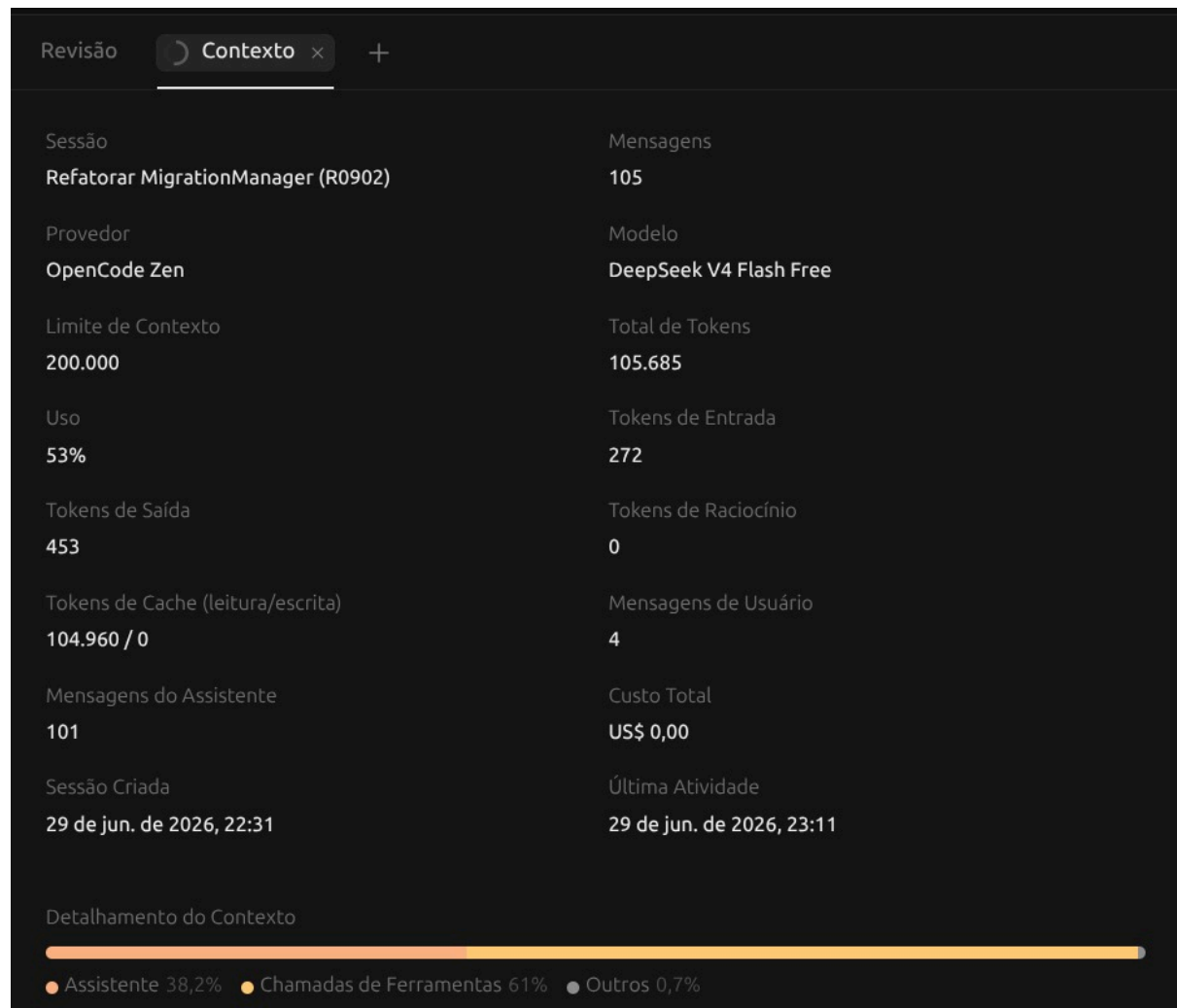
Tabela de identificação

module	obj	message
piccolo.apps.migrations.auto.migration_manager	MigrationManager	Too many instance attributes (16/7)

Seguiu o plano de refatoração: Não integralmente. O plano inicial alterava o construtor para usar meta e operations, mas isso exigiria mudar várias chamadas externas. Após os testes, foi adotado um `__init__` customizado compatível com os dois formatos, preservando o uso legado com `migration_id`, `app_name` e demais parâmetros diretos.

Link da publicação OpenCode: <https://opncd.ai/share/JLt1uLgH>

Print **consumo** **de** **token:**



Análise da refatoração: A refatoração valeu a pena, pois a LLM ajudou a identificar que a primeira solução era arriscada por quebrar compatibilidade com chamadas já existentes. Apesar do alto consumo, com 105.685 tokens, cerca de 53% do contexto, e duração de aproximadamente 40 minutos, o uso foi justificável porque a ocorrência era mais complexa e envolvia muitos pontos de uso. O custo foi maior que nas refatorações anteriores, mas compensou por evitar alterações em 34 chamadas e manter compatibilidade retroativa sem prejudicar o comportamento esperado do sistema.

3.2.5 Ocorrência Número 5

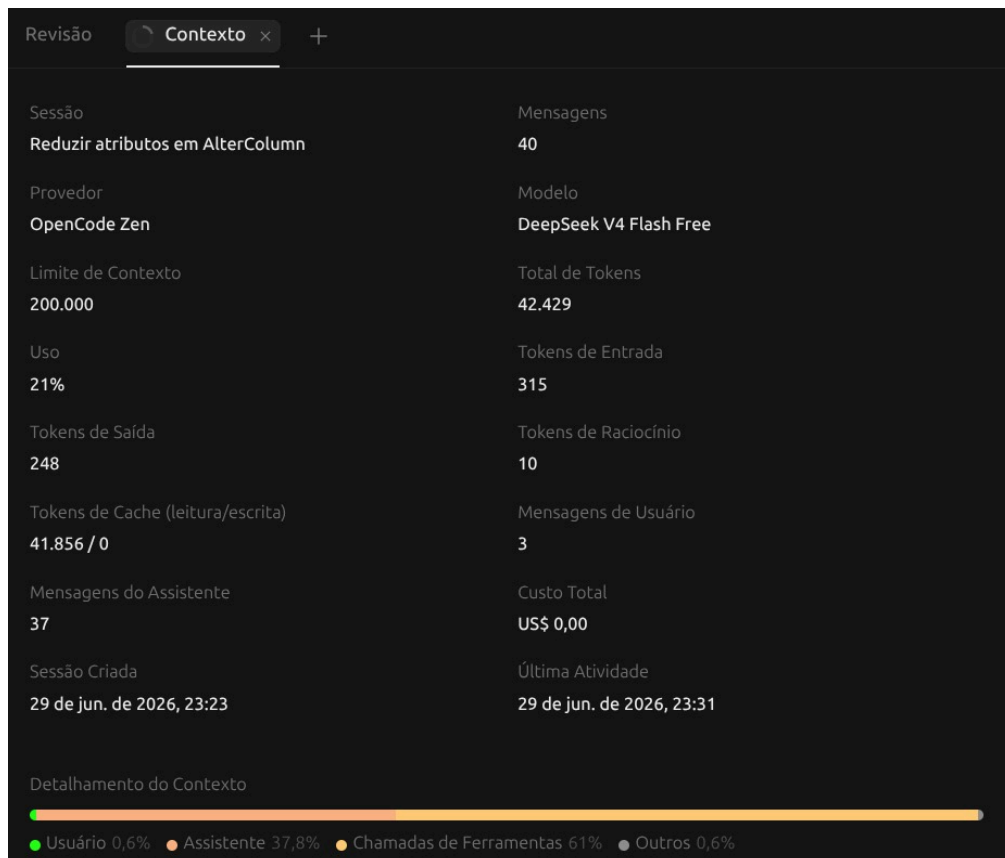
Tabela de identificação

module	obj	message
piccolo.apps.migrations.auto.operations	AlterColumn	Too many instance attributes (9/7)

Seguiu o plano de refatoração: Sim. O plano foi aceito de primeira, pois agrupava atributos relacionados de forma coerente, reduzia o smell sem mudar a assinatura pública de `manager.alter_column(...)` e mantinha o impacto limitado aos arquivos necessários.

Link da publicação OpenCode: <https://opncd.ai/share/12SIHXAU>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois corrigiu o excesso de atributos em AlterColumn sem tornar a solução muito invasiva. A sessão durou cerca de 8 minutos e consumiu 42.429 tokens, aproximadamente 21% do contexto. O custo foi moderado, mas aceitável, já que o plano foi seguido sem retrabalho e a solução manteve compatibilidade com o uso público do método.

3.2.6 Ocorrência Número 6

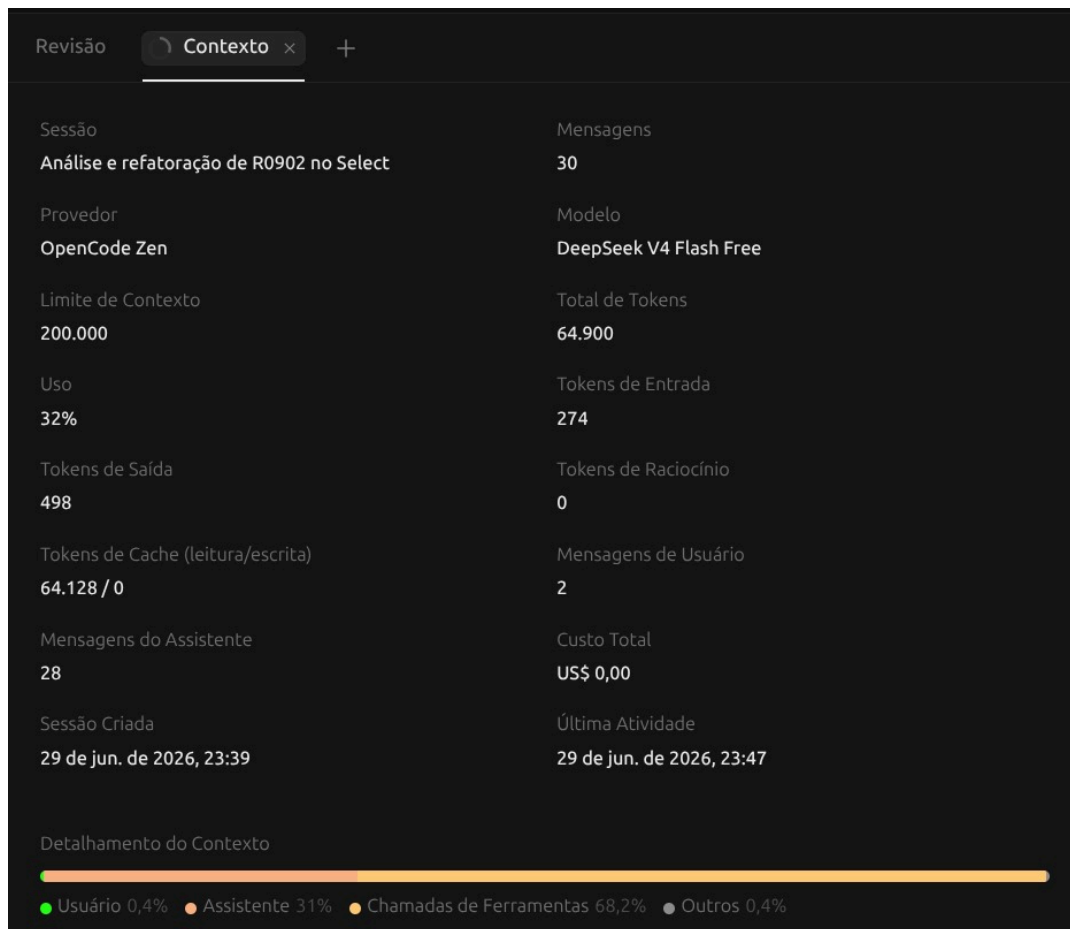
Tabela de identificação

module	obj	message
piccolo.query.methods.select	Select	Too many instance attributes (13/7)

Seguiu o plano de refatoração: Sim. O plano foi aceito de primeira, pois reduzia os atributos da classe sem alterar outros arquivos e ainda mantinha compatibilidade com a classe base Query por meio de propriedades getter e setter.

Link da publicação OpenCode: <https://opncd.ai/share/LLxjbXYL>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois corrigiu um smell estrutural em uma classe central de consulta sem modificar a API externa. A sessão durou cerca de 8 minutos e consumiu 64.900 tokens, cerca de 32% do contexto. O consumo foi relativamente alto, mas aceitável, já que o plano foi seguido sem retrabalho e a solução preservou o comportamento esperado, reduzindo a complexidade interna da classe Select.

3.2.7 Ocorrência Número 7

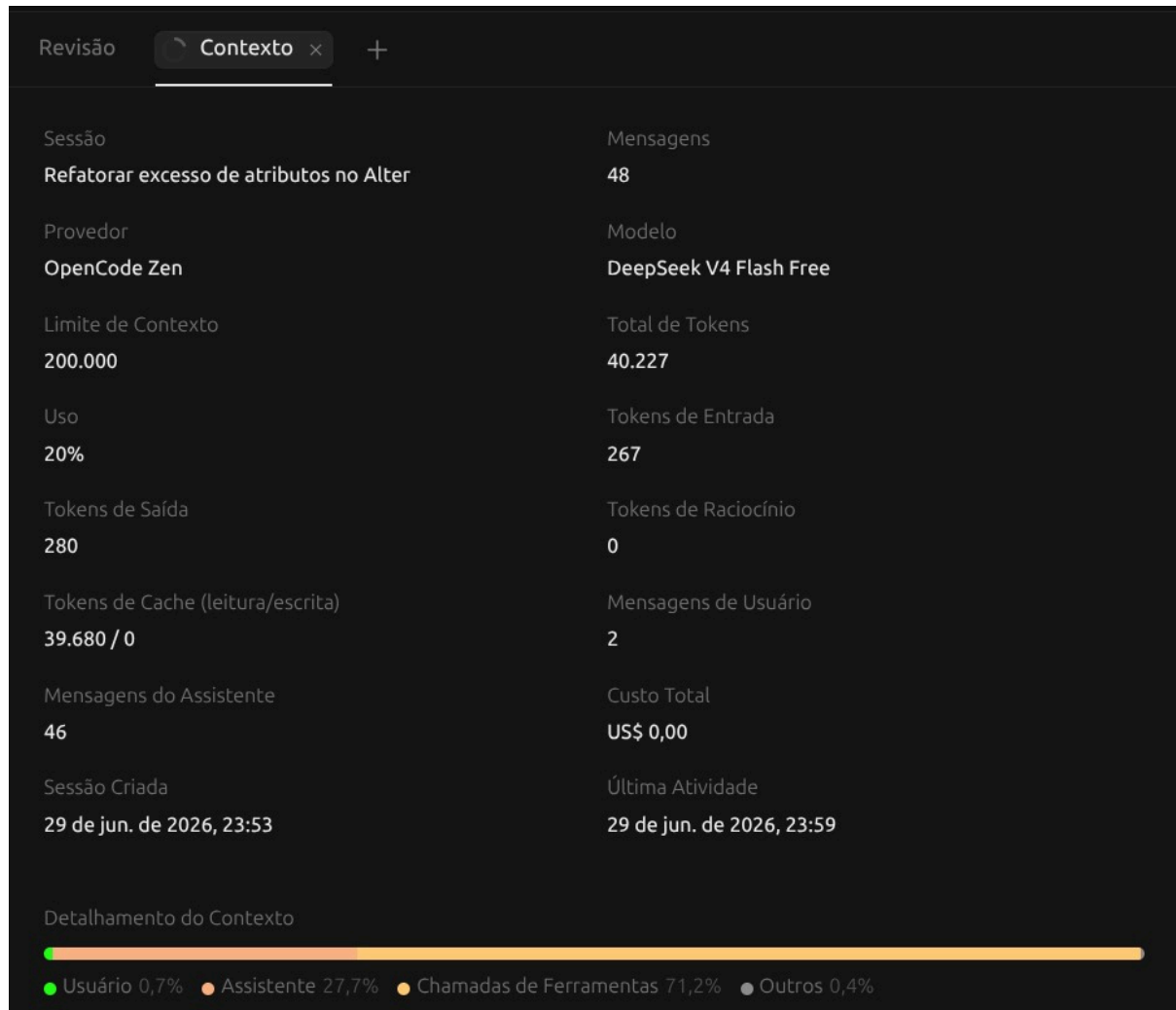
Tabela de identificação

module	obj	message
piccolo.query.methods.alter	Alter	Too many instance attributes (16/7)

Seguiu o plano de refatoração: Sim. O plano foi aceito de primeira, pois mantinha a API pública da classe, preservava os métodos existentes e concentrava a mudança apenas na organização interna dos atributos. Houve apenas um ajuste técnico: adicionar uma supressão local na dataclass `_AlterOperations`, por ela ser apenas um contêiner de dados.

Link da publicação OpenCode: <https://opncd.ai/share/R56jy0R3>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois reduziu a complexidade da classe `Alter` sem alterar seu comportamento externo. A sessão durou cerca de 6 minutos e consumiu 40.227 tokens, aproximadamente 20% do contexto. O custo foi moderado, mas aceitável, já que o plano foi seguido sem retrabalho relevante e a solução resolveu o smell mantendo a compatibilidade do código.

3.2.8 Ocorrência Número 8

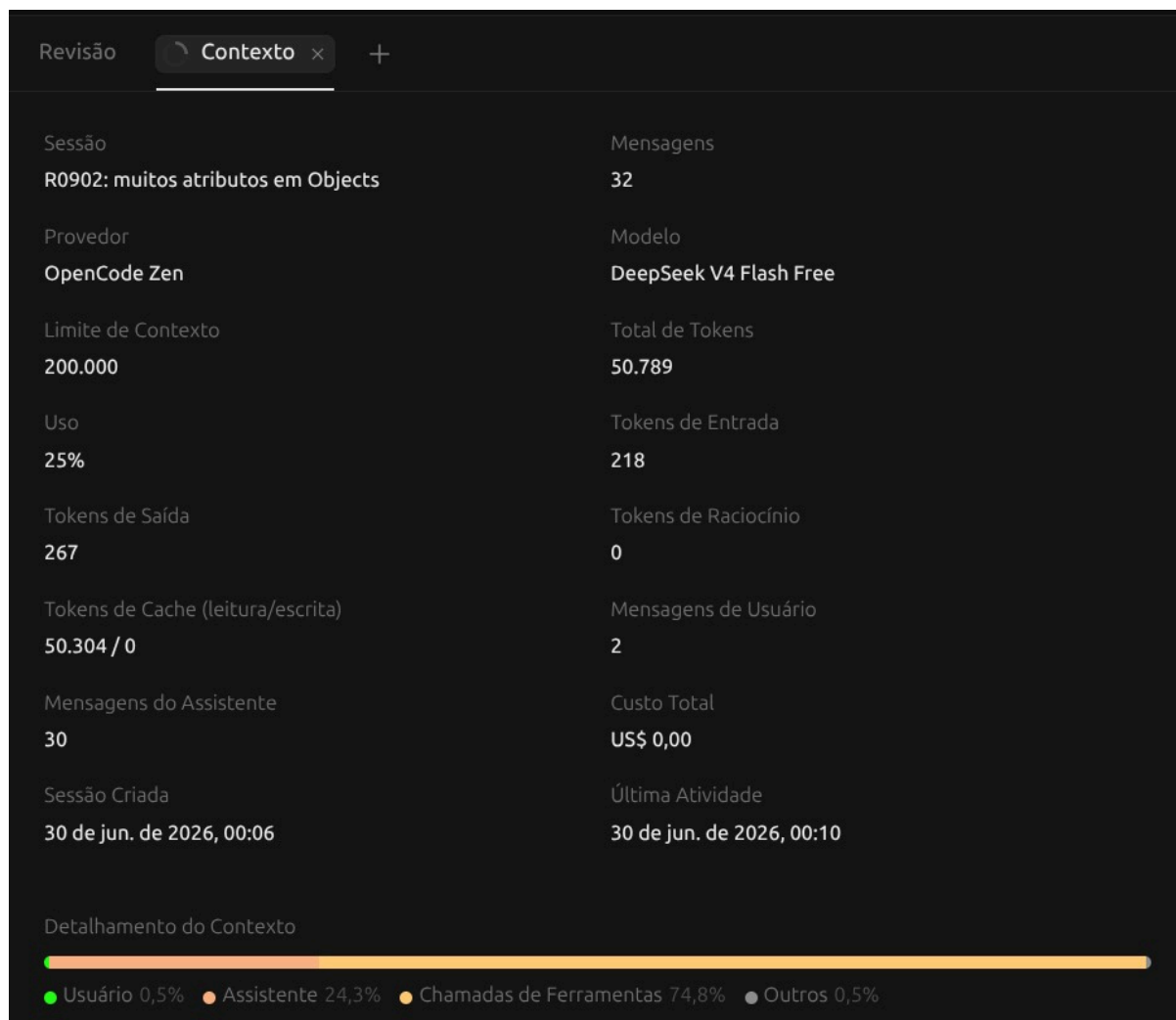
Tabela de identificação

module	obj	message
piccolo.query.methods.objects	Objects	Too many instance attributes (9/7)

Seguiu o plano de refatoração: Sim. O plano foi aceito de primeira, pois seguia o mesmo padrão já usado em Select, reduzia a quantidade de atributos e mantinha a compatibilidade por meio de propriedades getter e setter.

Link da publicação OpenCode: <https://opncd.ai/share/xjCoAHDk>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois resolveu o excesso de atributos em Objects sem alterar a API pública da classe. A sessão durou cerca de 4 minutos e consumiu 50.789 tokens, aproximadamente 25% do contexto. Apesar do consumo relativamente alto, o uso da LLM foi eficiente, já que o plano foi aceito sem retrabalho e a solução manteve o comportamento esperado do sistema.

3.2.9 Ocorrência Número 9

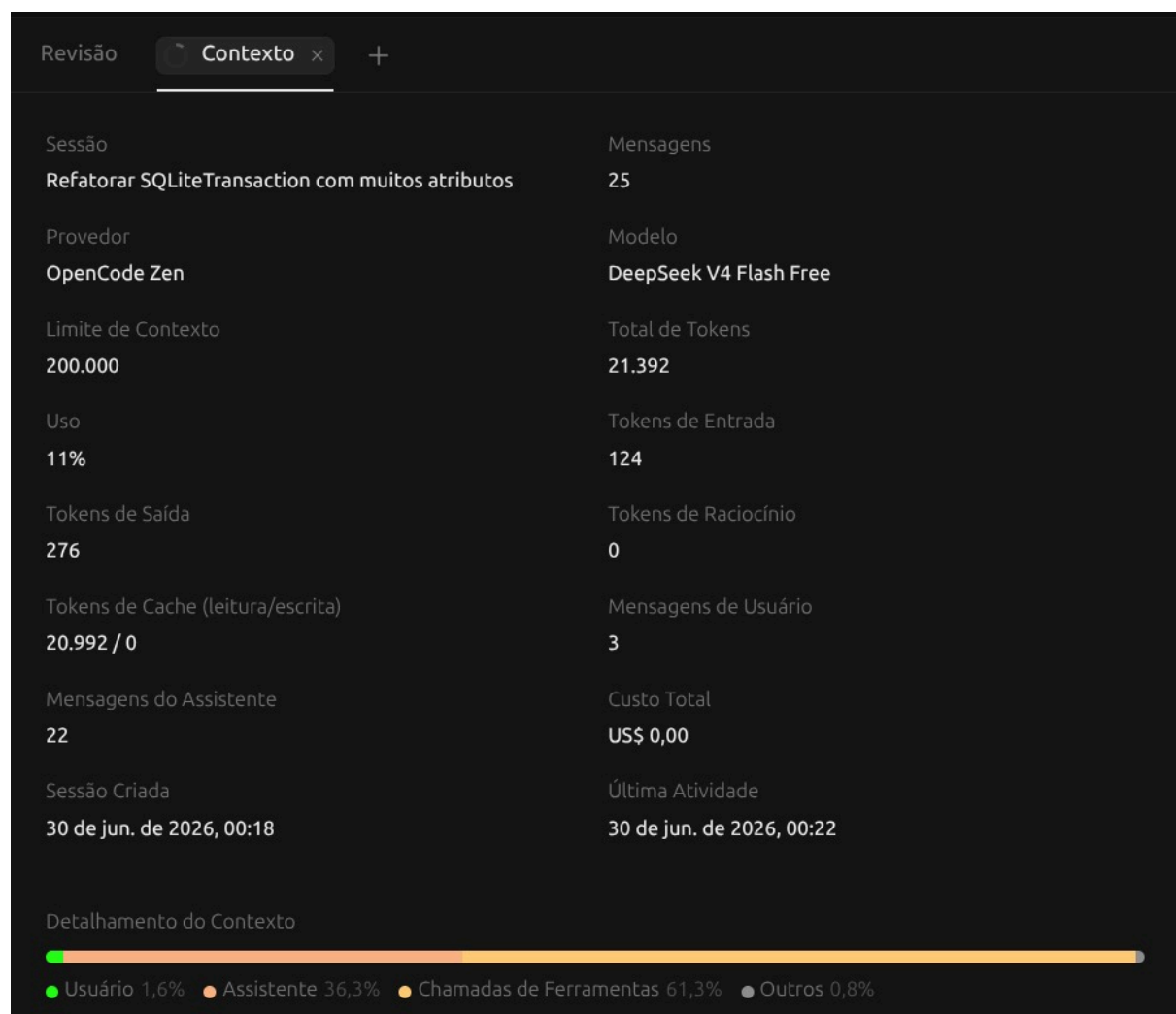
Tabela de identificação

module	obj	message
piccolo.engine.sqlite	SQLiteTransaction	Too many instance attributes (8/7)

Seguiu o plano de refatoração: Sim, com uma validação antes da implementação. Antes de seguir, foi confirmado que `transaction_type` não era código morto, pois é usado no método `begin()` para definir o tipo de transação SQLite. Depois disso, o plano original foi seguido.

Link da publicação OpenCode: <https://opncd.ai/share/1L9oiTEs>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois resolveu o smell com uma mudança pequena e segura. A sessão durou cerca de 4 minutos e consumiu 21.392 tokens, cerca de 11% do contexto. O custo foi baixo em comparação com outras ocorrências, e a solução melhorou a representação do estado da transação ao trocar dois booleanos mutuamente exclusivos por uma enum mais clara.

3.2.10 Ocorrência Número 10

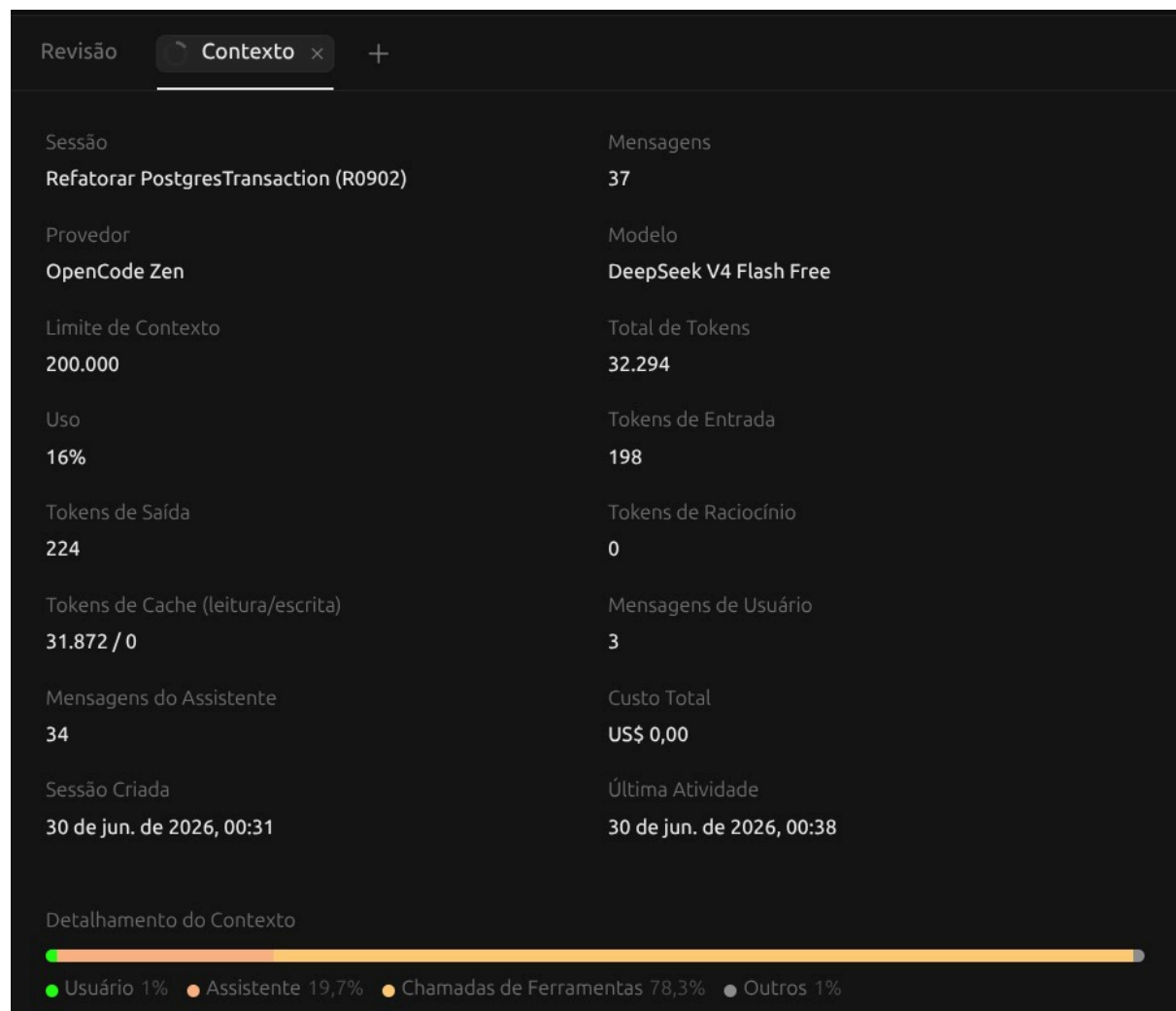
Tabela de identificação

module	obj	message
piccolo.engine.postgres	PostgresTransaction	Too many instance attributes (8/7)

Seguiu o plano de refatoração: Parcialmente. O plano inicial previa criar a enum diretamente em postgres.py, mas foi ajustado para centralizar TransactionState em base.py. A mudança foi melhor porque corrigiu o smell e ainda melhorou a organização do código.

Link da publicação OpenCode: <https://opncd.ai/share/rTKcBnXY>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, reduzindo a classe de 8 para 7 atributos, e ainda eliminou duplicação entre os engines SQLite e Postgres. A sessão durou cerca de 7 minutos e

consumiu 32.294 tokens, aproximadamente 16% do contexto. O custo foi moderado, mas compensou pela solução mais limpa e reutilizável.

3.2.11 Ocorrência Número 11

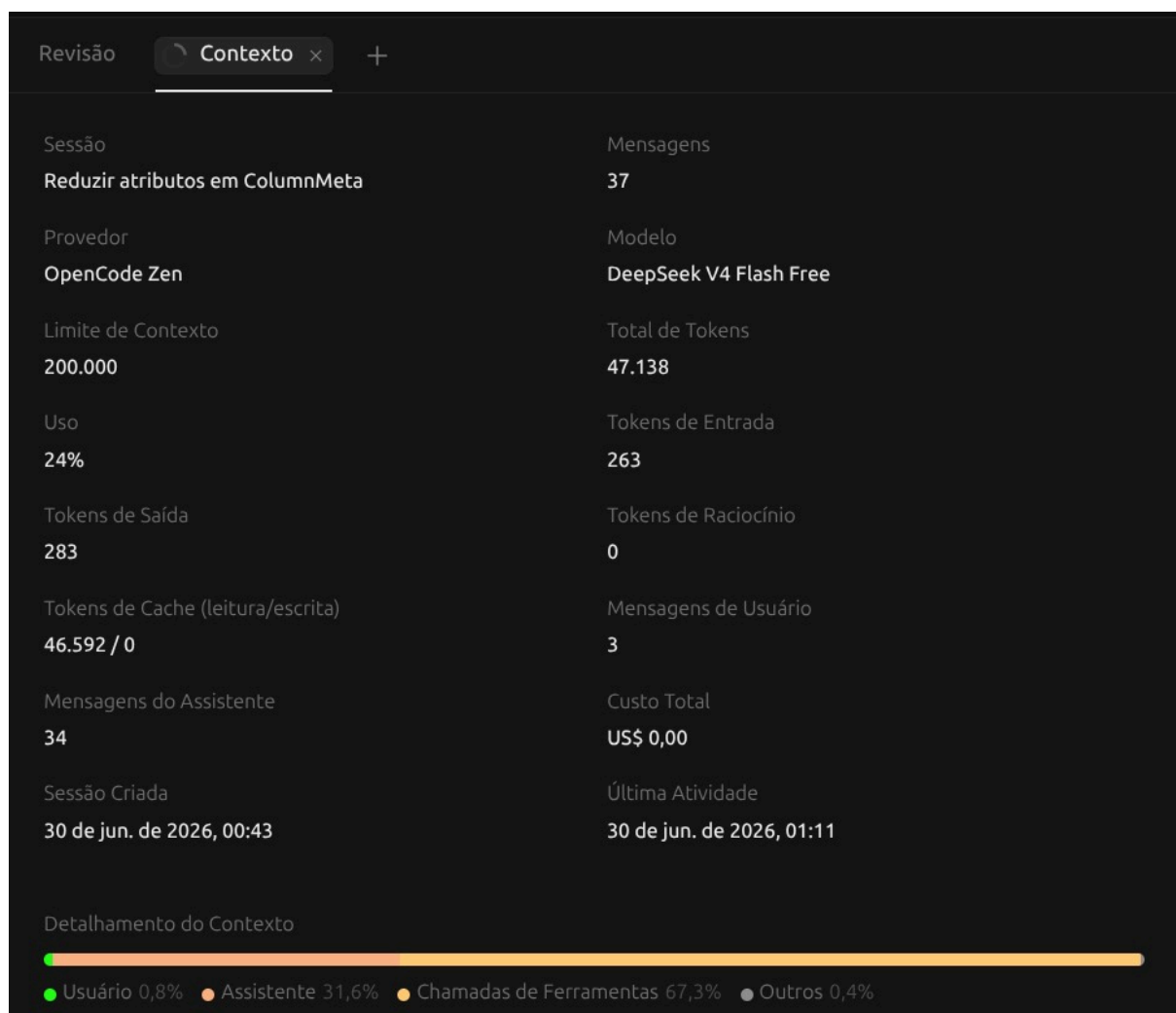
Tabela de identificação

module	obj	message
piccolo.columns.base	ColumnMeta	Too many instance attributes (15/7)

Seguiu o plano de refatoração: Parcialmente. A primeira proposta da LLM ficou indecisa e chegou a considerar `__getattr__`, mas essa abordagem foi descartada. O plano final seguiu o padrão já validado no projeto: sub-dataclass com propriedades getter e setter, preservando acessos como `col._meta.null` e `col._meta.unique`.

Link da publicação OpenCode: <https://opncd.ai/share/Wbjsa0kx>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois resolveu um smell relevante em uma classe bastante usada sem quebrar a API existente. A sessão durou cerca de 28 minutos e consumiu 47.138 tokens, aproximadamente 24% do contexto. O custo foi moderado, principalmente porque foi necessário corrigir a direção inicial do plano, mas compensou pela solução final mais segura, compatível e alinhada ao padrão do projeto.

3.3 too-many-branches

Quantidade de ocorrências: 13

3.2.1 Ocorrência Número 1

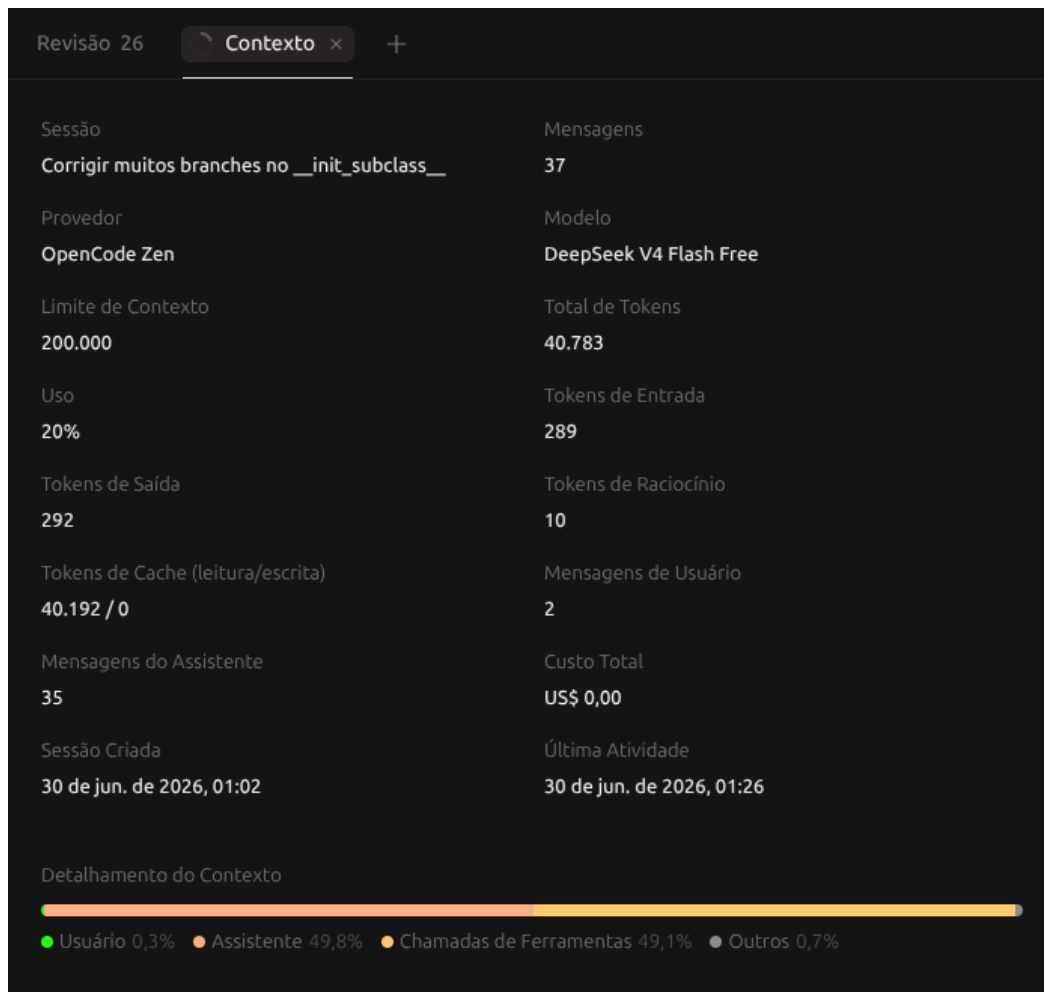
Tabela de identificação

module	obj	message
piccolo.table	Table.__init_subclass__	Too many branches (17/12)

Seguiu o plano de refatoração: Sim. O primeiro plano foi seguido, pois ele reduzia a complexidade do método sem alterar o comportamento esperado e ainda organizava melhor as responsabilidades internas da classe.

Link da publicação OpenCode: <https://opncd.ai/share/IPw61gla>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois o método tinha 17 branches para um limite de 12, indicando excesso de lógica concentrada em uma única função. A sessão durou cerca de 24 minutos e consumiu 40.783 tokens, aproximadamente 20% do contexto. O custo foi moderado, mas aceitável, já que o plano foi seguido sem retrabalho e a solução tornou o código mais organizado, legível e fácil de manter.

3.2.2 Ocorrência Número 2

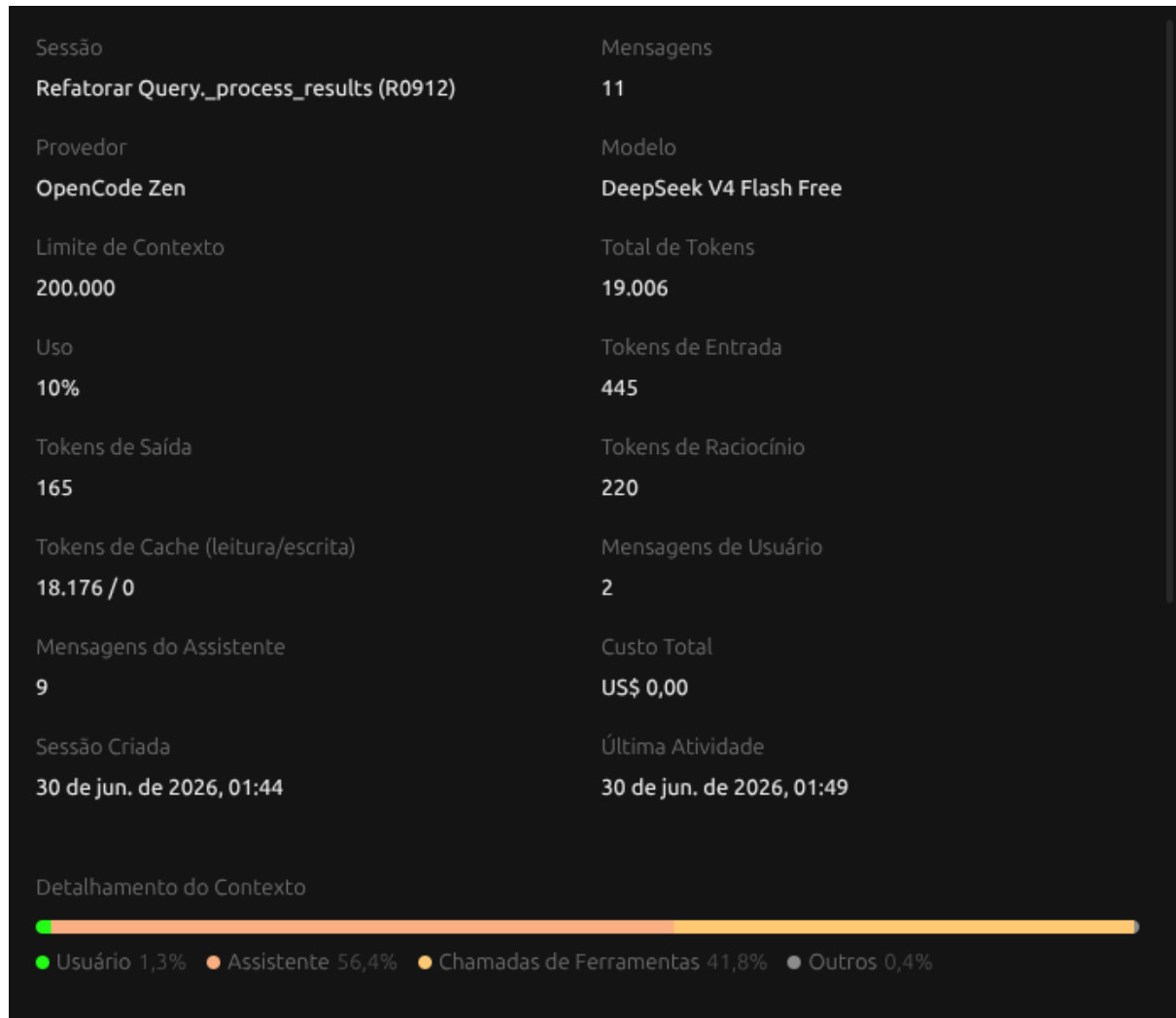
Tabela de identificação

module	obj	message
piccolo.query.base	Query._process_results	Too many branches (19/12)

Seguiu o plano de refatoração: Sim. O plano foi aceito de primeira, pois propunha uma divisão coerente da lógica do método sem alterar a API pública nem o comportamento esperado da consulta.

Link da publicação OpenCode: <https://opncd.ai/share/tksOWRJS>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois o problema era de excesso de branches em um método central de processamento de resultados. A sessão durou cerca de 5 minutos e consumiu 19.006 tokens, aproximadamente 10% do contexto. O custo foi baixo em relação às outras ocorrências, e como o plano foi aceito sem retrabalho, o uso da LLM foi eficiente para organizar a lógica e reduzir a complexidade do método.

3.2.3 Ocorrência Número 3

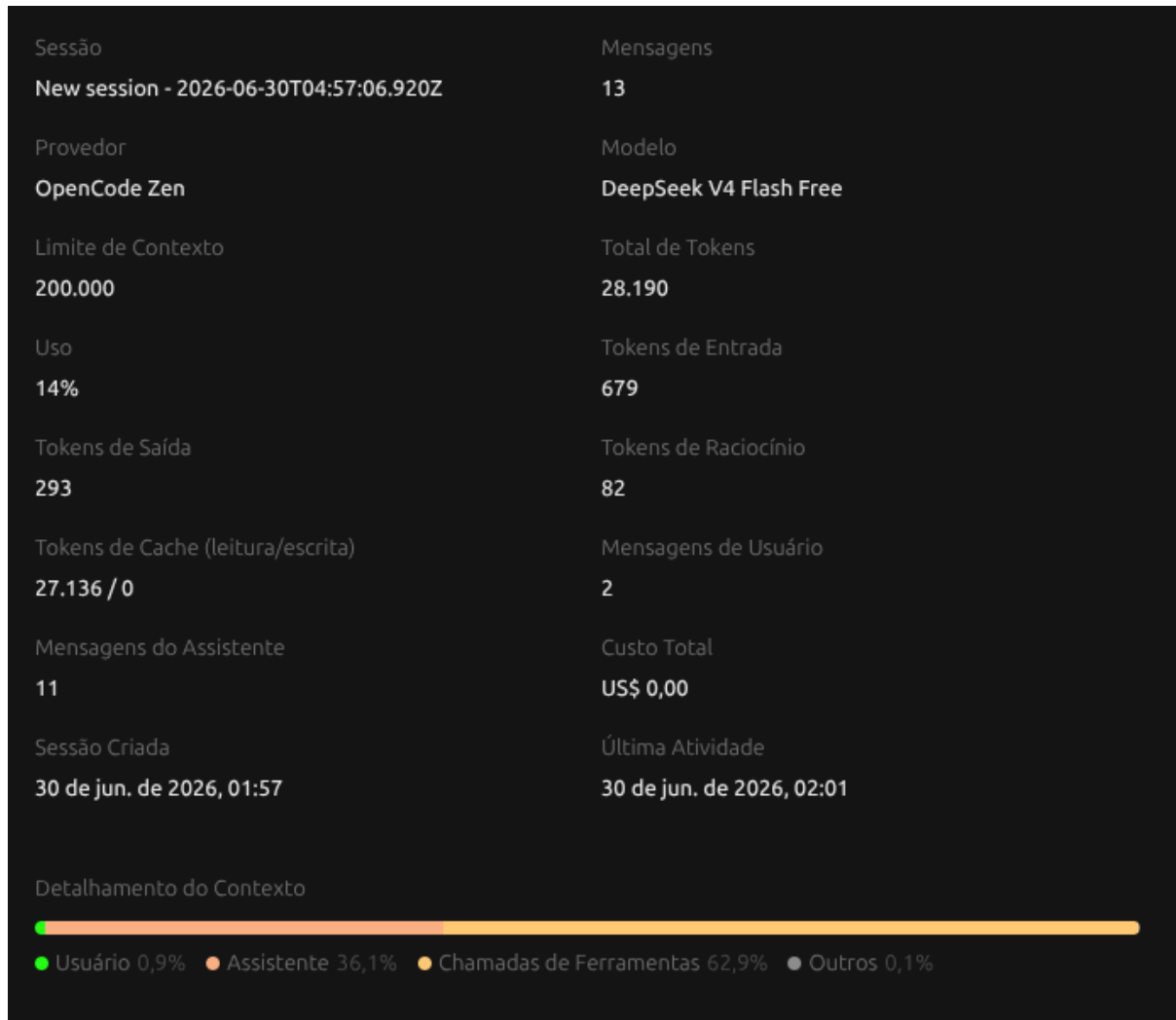
Tabela de identificação

module	obj	message
piccolo.query.methods.select	Select.response_handler	Too many branches (19/12)

Seguiu o plano de refatoração: Sim. O plano foi aceito de primeira, pois separava corretamente a lógica por engine, reduzia os branches do método principal e mantinha o comportamento esperado sem alterar a API pública.

Link da publicação OpenCode: <https://opncd.ai/share/9t7G6qwY>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois response_handler tinha 19 branches para um limite de 12, tornando o método difícil de manter. A sessão durou cerca de 4 minutos e consumiu 28.190 tokens, aproximadamente 14% do contexto. O custo foi baixo/moderado e compensou, já que o plano foi seguido sem retrabalho e a lógica ficou melhor organizada em métodos específicos para cada engine.

3.2.4 Ocorrência Número 4

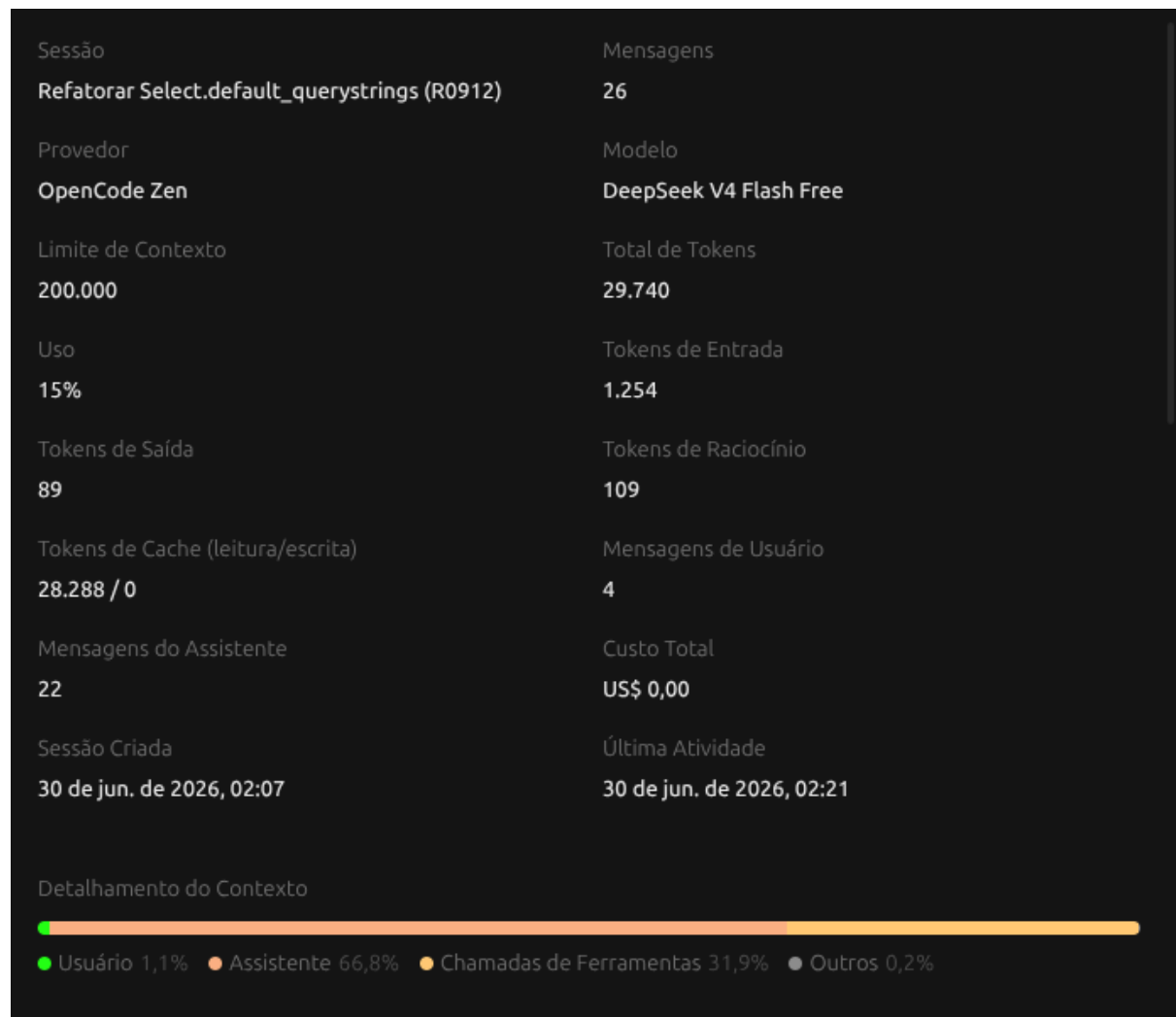
Tabela de identificação

module	obj	message
piccolo.query.methods.select	Select.default_querystrings	Too many branches (14/12)

Seguiu o plano de refatoração: Não integralmente. O primeiro plano foi aceito, mas causou falhas nos testes com SQL inválido, como erros próximos de LIMIT,) e ORDER BY. Por isso, foi necessário solicitar um novo plano, que corrigiu a condição do order_by e preservou melhor o comportamento original.

Link da publicação OpenCode: <https://opncd.ai/share/P54Lpx7s>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, mas exigiu retrabalho. A sessão durou cerca de 14 minutos e consumiu 29.740 tokens, cerca de 15% do contexto. O custo foi moderado, principalmente porque a primeira solução reduziu o smell, mas quebrou testes ao tratar order_by de forma diferente do código original. Ainda assim, o uso da LLM foi útil porque ajudou a identificar a

causa do erro e chegar a uma solução final mais segura, mantendo a redução de branches sem alterar a montagem correta da query.

3.2.5 Ocorrência Número 5

Tabela de identificação

module	obj	message
piccolo.columns.base	Column.get_sql_value	Too many branches (15/12)

Seguiu o plano de refatoração: Sim. O plano foi aceito e seguido, pois dividia o método por responsabilidade, mantinha o comportamento esperado e preservava a API pública de `get_sql_value`.

Link da publicação OpenCode: <https://opncd.ai/share/CQgh1ca3>

Print consumo de token:

Sessão	Mensagens
Reduzir branches em <code>Column.get_sql_value</code>	19
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	26.967
Uso	Tokens de Entrada
13%	52
Tokens de Saída	Tokens de Raciocínio
141	22
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
26.752 / 0	2
Mensagens do Assistente	Custo Total
17	US\$ 0,00
Sessão Criada	Última Atividade
30 de jun. de 2026, 02:31	30 de jun. de 2026, 02:36
Detalhamento do Contexto Assistente 46,2% Chamadas de Ferramentas 51,9% Outros 1,9%	

Análise da refatoração: A refatoração valeu a pena, pois `get_sql_value` tinha 15 branches para um limite de 12, e a extração da lógica por engine tornou o método mais simples e organizado. A sessão durou cerca de 5 minutos e consumiu 26.967 tokens, aproximadamente 13% do contexto. O custo foi baixo/moderado e aceitável, já que o plano foi seguido sem retrabalho e a solução resolveu o R0912 mantendo os testes aplicáveis funcionando.

3.2.6 Ocorrência Número 6

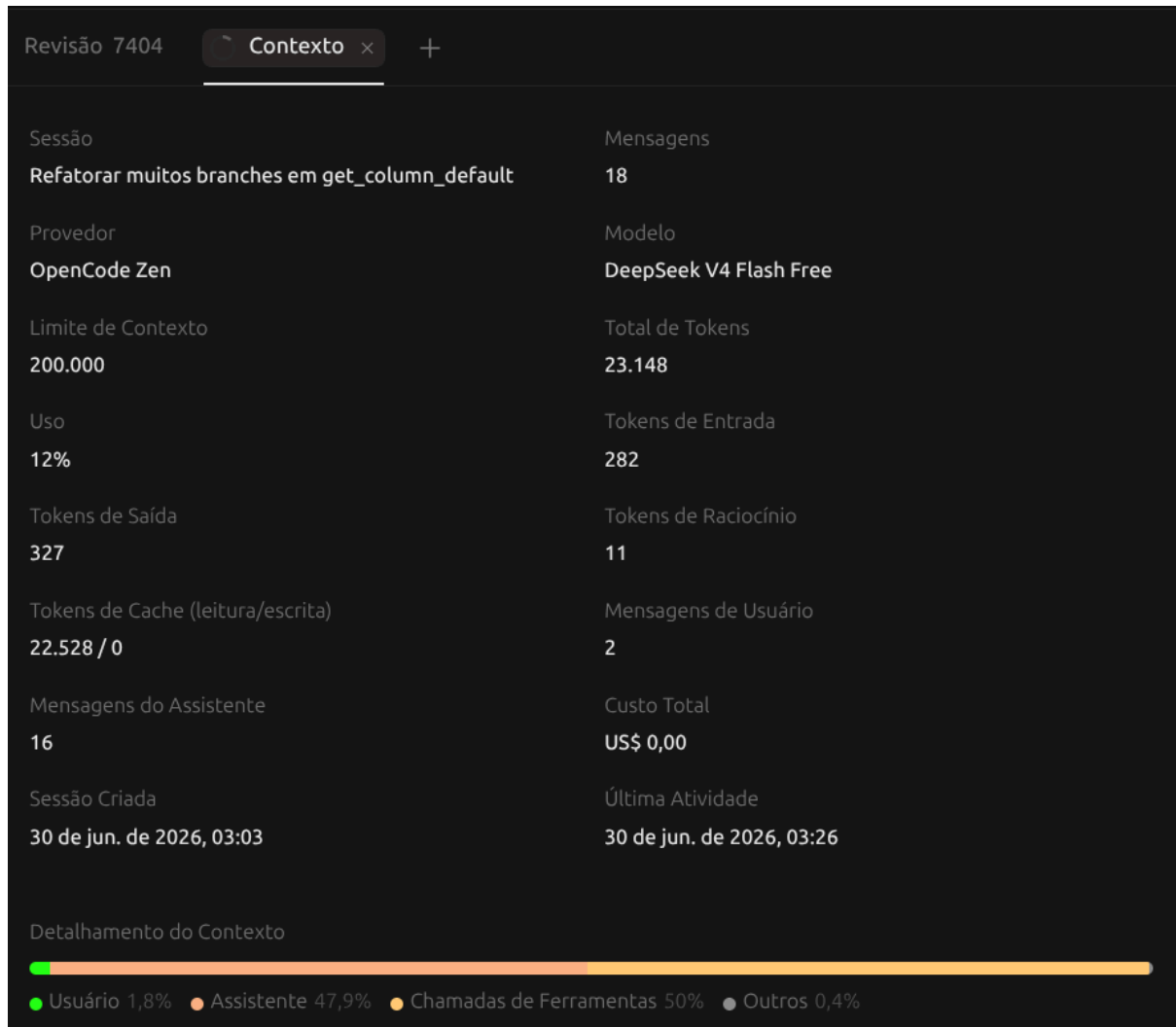
Tabela de identificação

module	obj	message
piccolo.apps.schema.commands.generate	get_column_default	Too many branches (17/12)

Seguiu o plano de refatoração: Sim. O plano foi aceito e seguido, pois dividia o método por responsabilidade, mantinha o comportamento esperado e preservava a API pública de `get_sql_value`.

Link da publicação OpenCode: <https://opncd.ai/share/IPw61gla>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois corrigiu o smell R0912: too-many-branches na função `get_column_default`, reduzindo a contagem de 17 para aproximadamente 5 branches através da substituição da cadeia `if/elif/else` por uma `dispatch table`. A sessão durou cerca de 23 minutos e consumiu 23.148 tokens, aproximadamente 12% do contexto, com custo de US\$ 0,00 utilizando o modelo DeepSeek V4 Flash Free. O plano foi seguido sem retrabalho, e a solução melhorou a extensibilidade do código, isolando cada tipo de coluna em um handler dedicado e facilitando a adição de novos tipos sem aumentar a complexidade ciclomática da função principal.

3.2.7 Ocorrência Número 7

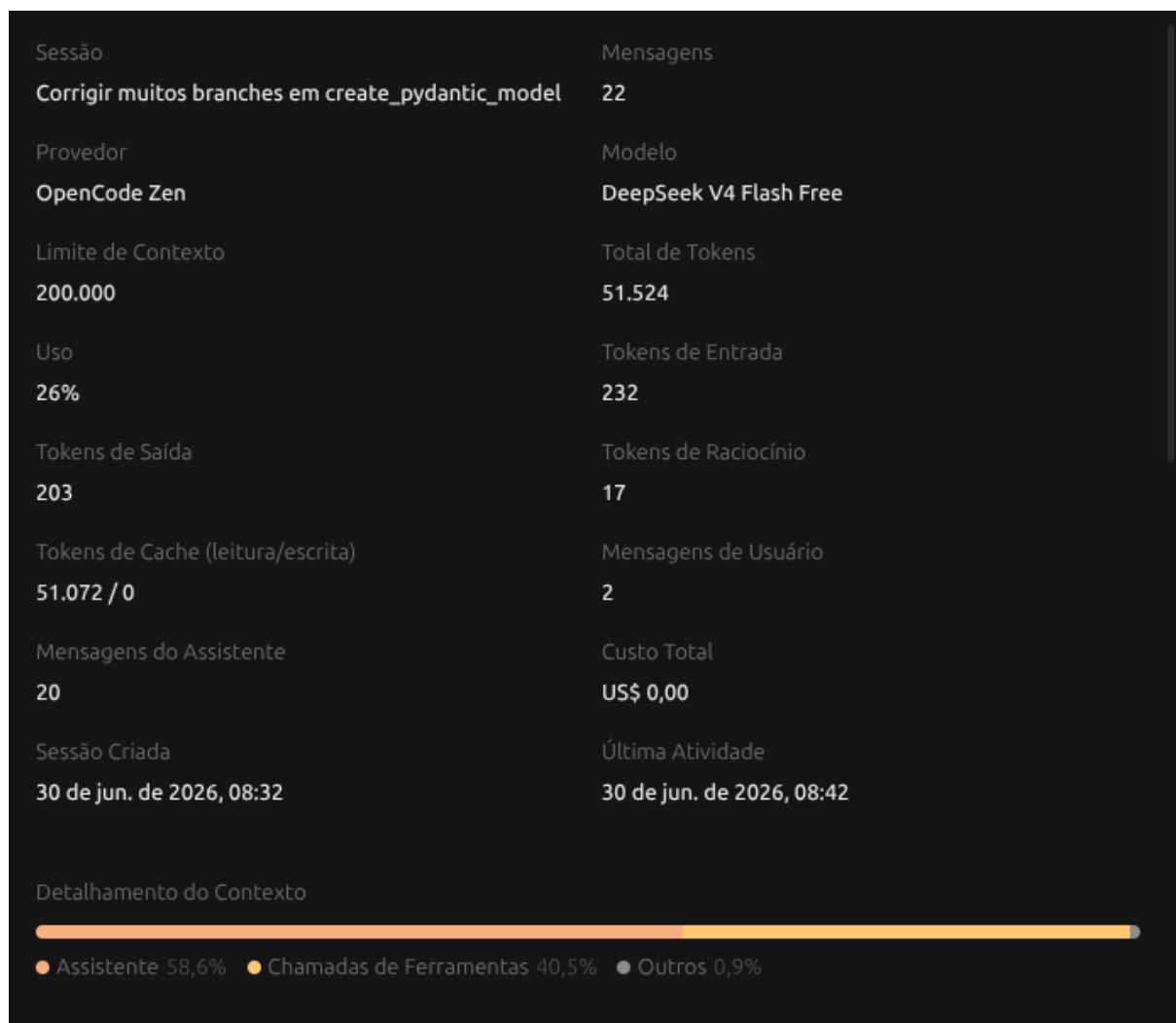
Tabela de identificação

module	obj	message
piccolo.utils.pydantic	create_pydantic_model	Too many branches (23/12)

Seguiu o plano de refatoração: Sim. O plano foi aceito de primeira, pois separava responsabilidades claras da função principal, reduzia os branches e mantinha a API pública de `create_pydantic_model` inalterada.

Link da publicação OpenCode: <https://opncd.ai/share/4u11tmr0>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois `create_pydantic_model` tinha 23 branches para um limite de 12, tornando a função difícil de manter. A sessão durou cerca de 10 minutos e consumiu 51.524 tokens, aproximadamente 26% do contexto. Apesar do consumo moderado/alto, o uso da LLM compensou, já que o plano foi seguido sem retrabalho e a função principal caiu para cerca de 7 branches, ficando abaixo do limite do Pylint. Além disso, 39 testes passaram, com apenas uma falha preexistente de permissão no PostgreSQL, não relacionada à mudança.

3.2.8 Ocorrência Número 8

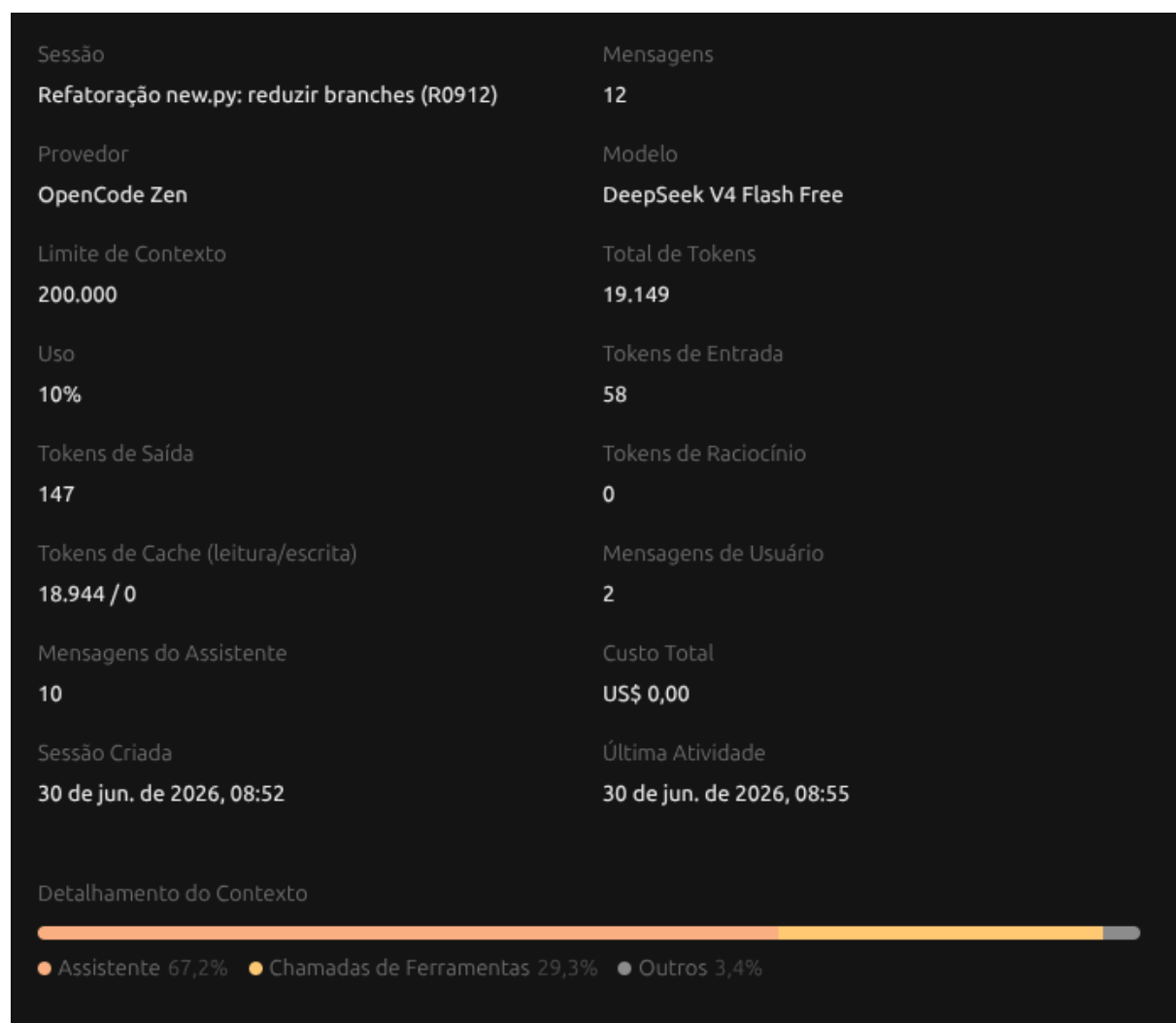
Tabela de identificação

module	obj	message
piccolo.apps.asgi.commands.new	new	Too many branches (15/12)

Seguiu o plano de refatoração: Sim. O plano foi seguido, pois separava corretamente as responsabilidades, reduzia os branches da função principal e mantinha o comportamento original do comando.

Link da publicação OpenCode: <https://opncd.ai/share/KOCJw9pC>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois a função new tinha 15 branches para um limite de 12, e a solução reduziu a função principal para apenas 2 branches. A sessão durou cerca de

3 minutos e consumiu 19.149 tokens, aproximadamente 10% do contexto. O custo foi baixo em comparação com outras ocorrências, e como o plano foi seguido sem retrabalho, o uso da LLM foi eficiente para corrigir o R0912 e melhorar a organização do código.

3.2.9 Ocorrência Número 9

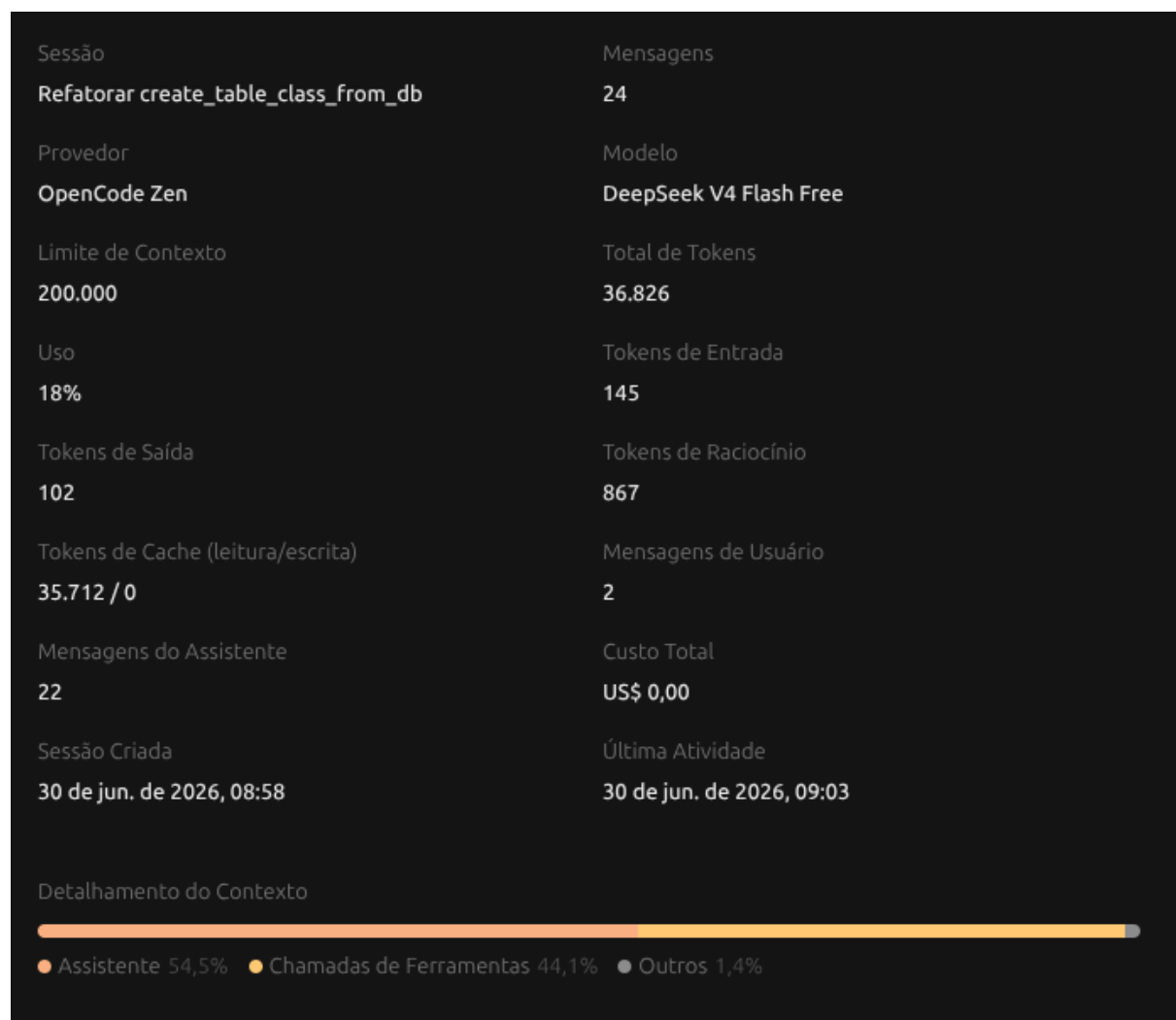
Tabela de identificação

module	obj	message
piccolo.apps.schema.commands.generate	create_table_class_from_db	Too many branches (22/12)

Seguiu o plano de refatoração: Sim. O plano foi seguido, pois reduzia diretamente os branches da função principal e separava responsabilidades sem alterar o objetivo da geração da classe.

Link da publicação OpenCode: <https://opncd.ai/share/26BszyK4>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois a função tinha 22 branches para um limite de 12, indicando excesso de decisões em um único método. A sessão durou cerca de 5 minutos e consumiu 36.826 tokens, aproximadamente 18% do contexto. O custo foi moderado, mas aceitável, já que o plano foi seguido e o R0912 foi resolvido. Porém, como surgiu um novo alerta em função auxiliar, o ganho foi parcial e pode exigir uma refatoração complementar.

3.2.10 Ocorrência Número 10

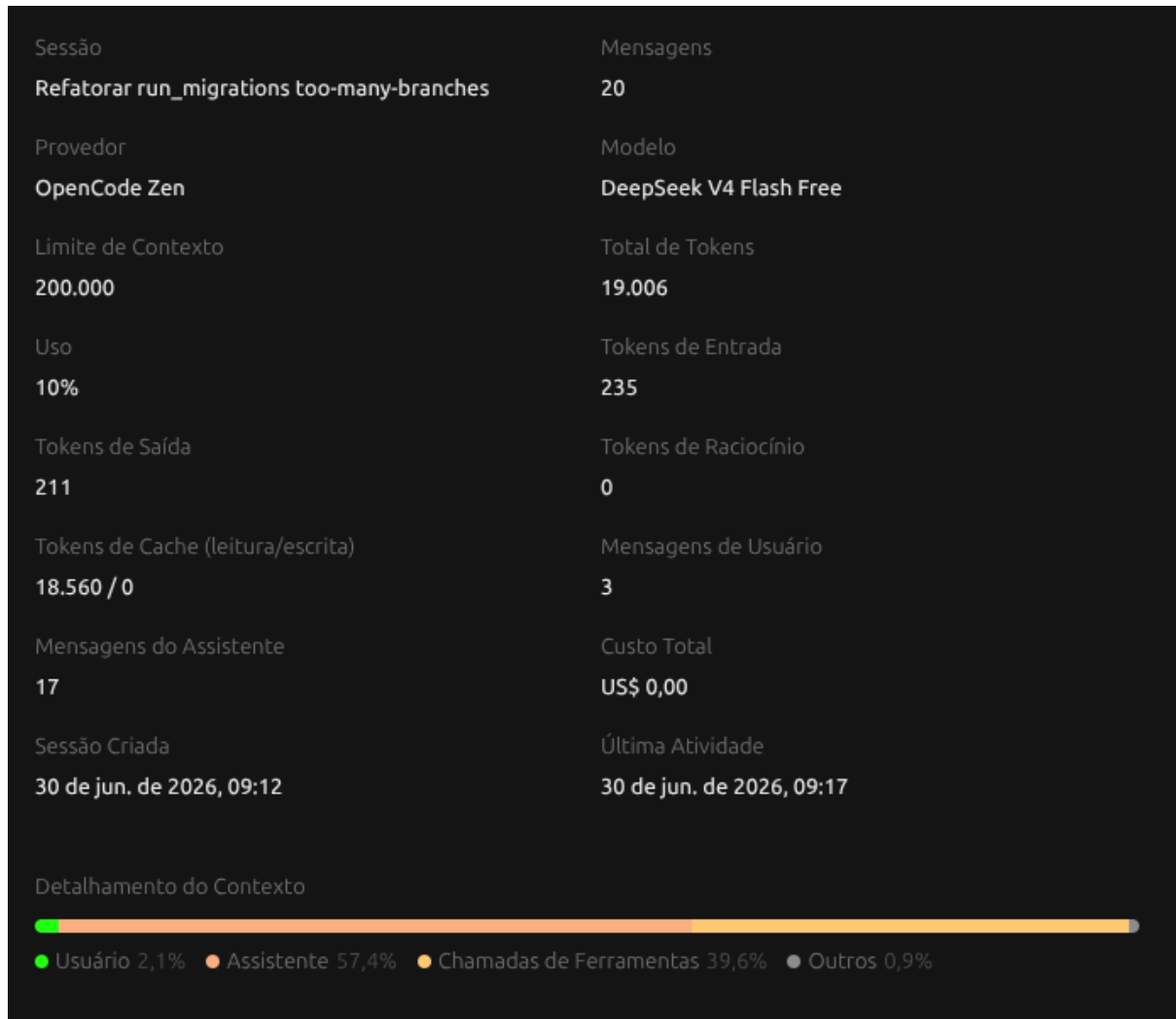
Tabela de identificação

module	obj	message
piccolo.apps.migrations.commands.forwards	ForwardsMigrationManager.run_migrations	Too many branches (14/12)

Seguiu o plano de refatoração: Sim, com alteração. O plano inicial foi ajustado porque retornar uma lista vazia para um migration_id inválido mudaria o comportamento do sistema. A versão final manteve a falha explícita com success=False, corrigindo o R0912 sem quebrar a lógica original.

Link da publicação OpenCode: <https://opncd.ai/share/qX6Jy2st>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois run_migrations tinha 14 branches para um limite de 12, e a solução reduziu o método principal para cerca de 3 branches. A sessão durou cerca de 5 minutos e consumiu 19.006 tokens, aproximadamente 10% do contexto. O custo foi baixo, e a alteração no plano foi positiva porque evitou uma regressão funcional: migration_id inválido continuou retornando erro, como no comportamento original.

3.2.11 Ocorrência Número 11

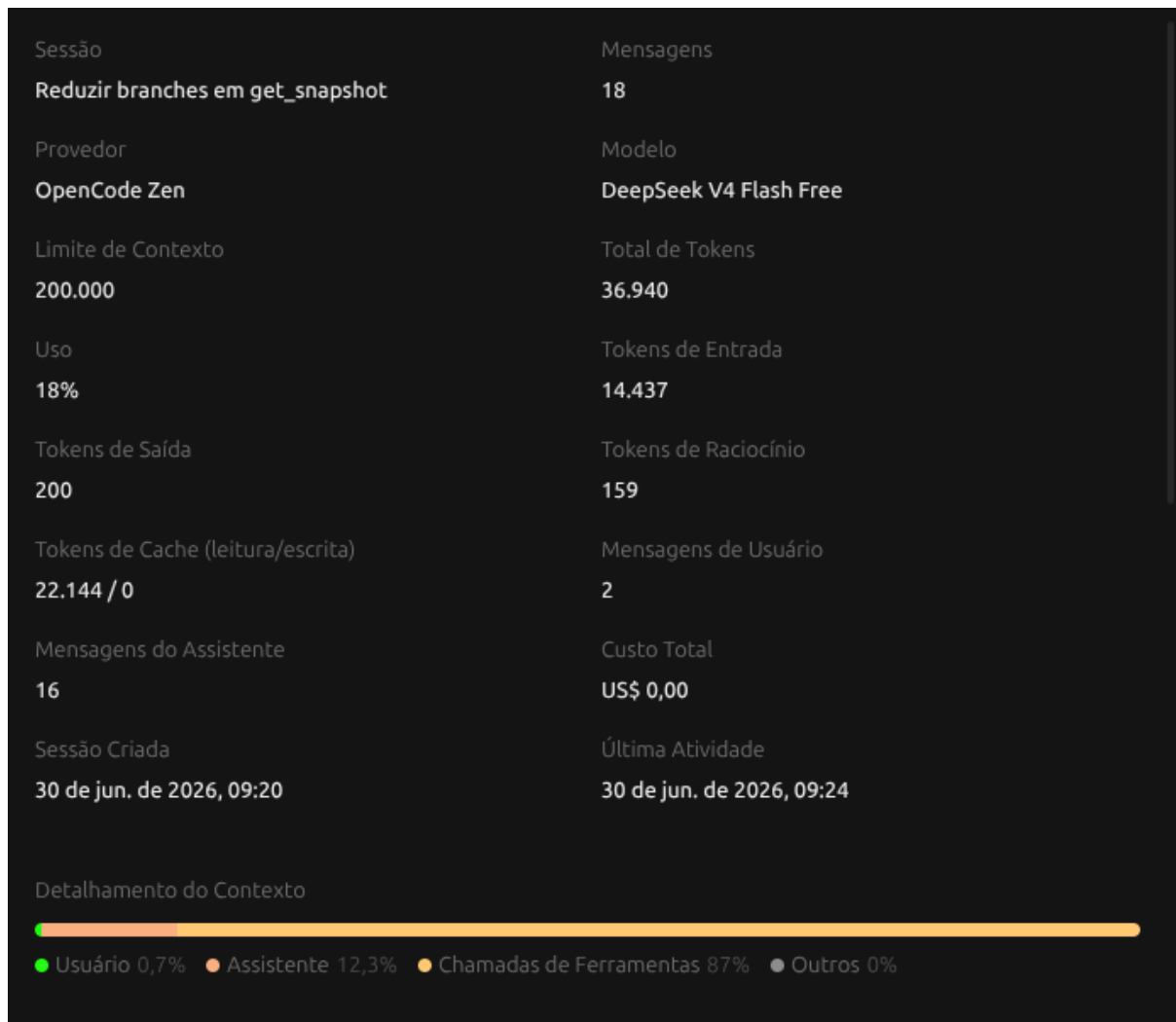
Tabela de identificação

module	obj	message
piccolo.apps.migrations.auto.schema_snapshot	SchemaSnapshot.get_snapshot	Too many branches (19/12)

Seguiu o plano de refatoração: Sim. O plano foi seguido, pois separava corretamente as responsabilidades do método, reduzia os branches de `get_snapshot` e mantinha a API pública sem alterações.

Link da publicação OpenCode: <https://opncd.ai/share/XFqjJM6S>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois `get_snapshot` tinha 19 branches para um limite de 12, concentrando muitas operações diferentes em um único método. A sessão durou cerca de 4 minutos e consumiu 36.940 tokens, aproximadamente 18% do contexto. O custo foi moderado, mas aceitável, já que a solução deixou o método principal mais simples, legível e organizado, delegando cada tipo de alteração para helpers específicos.

3.2.12 Ocorrência Número 12

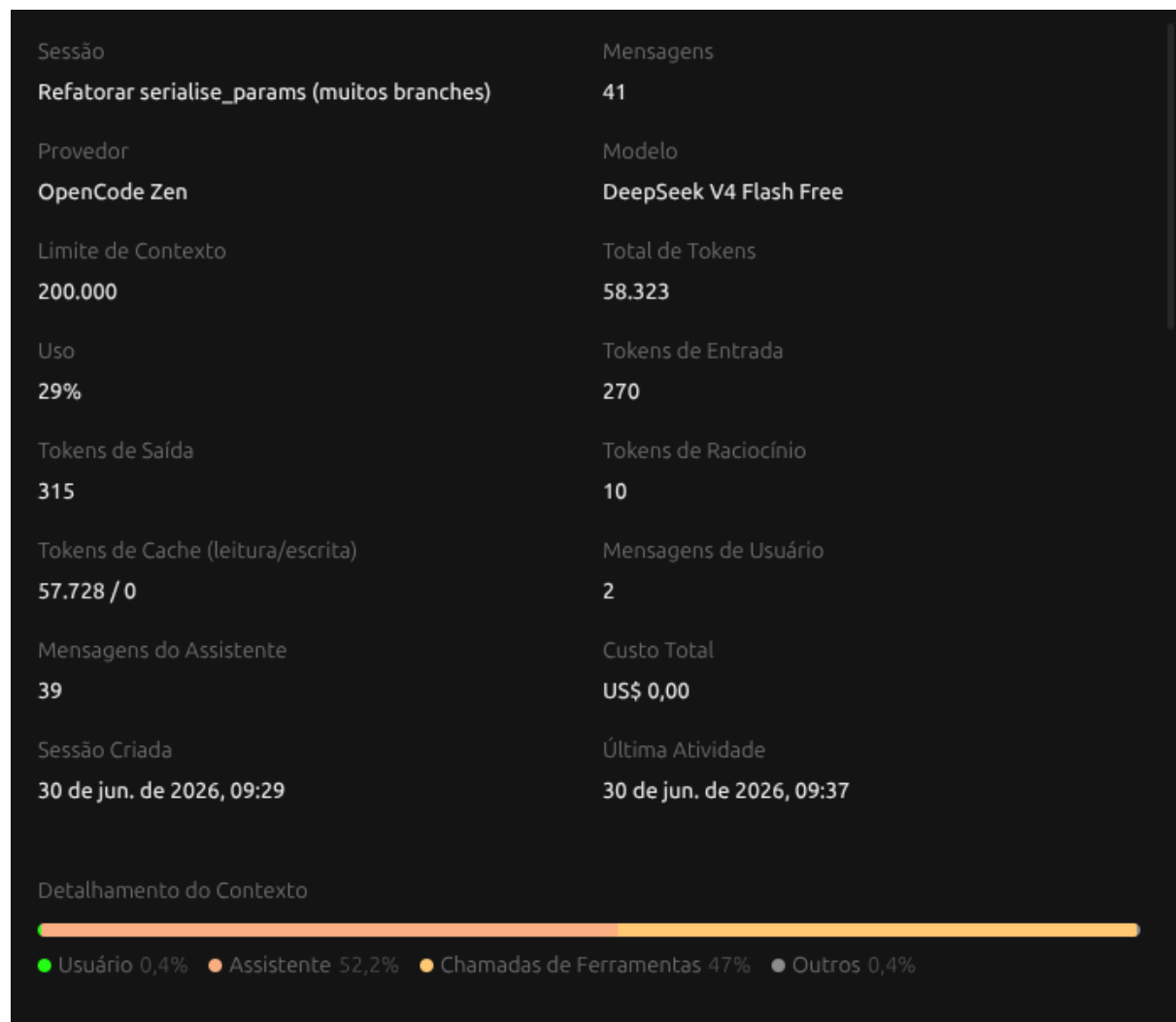
Tabela de identificação

module	obj	message
piccolo.apps.migrations.auto.serialisation	serialise_params	Too many branches (21/12)

Seguiu o plano de refatoração: Sim. O plano foi seguido, pois reduzia os branches sem alterar a lógica de serialização. A solução manteve cada caso de tratamento isolado em uma função específica e deixou serialise_params apenas coordenando o loop de dispatch.

Link da publicação OpenCode: <https://opncd.ai/share/E2l4hA2X>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena, pois serialise_params tinha 21 branches para um limite de 12. Após a mudança, a função principal ficou com apenas 3 branches, e os 22 testes do

módulo passaram, com Pylint 10/10 sem R0912. A sessão durou cerca de 8 minutos e consumiu 58.323 tokens, aproximadamente 29% do contexto. O custo foi moderado/alto, mas compensou pela melhoria estrutural e pela validação com testes.

3.2.13 Ocorrência Número 13

Tabela de identificação

module	obj	message
piccolo.apps.migrations.auto.migration_manager	MigrationManager._run_alter_columns	Too many branches (23/12)

Seguiu o plano de refatoração: Sim. O plano foi seguido, pois separava corretamente as responsabilidades e reduzia os branches do método principal sem alterar a lógica esperada. A refatoração também manteve os comentários e o comportamento original.

Link da publicação OpenCode: <https://opncd.ai/share/mBDeJPcw>

Print consumo de token:

Sessão	Mensagens
Refatoração _run_alter_columns (too-many-branches)	29
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	42.901
Uso	Tokens de Entrada
21%	506
Tokens de Saída	Tokens de Raciocínio
395	16
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
41.984 / 0	2
Mensagens do Assistente	Custo Total
27	US\$ 0,00
Sessão Criada	Última Atividade
30 de jun. de 2026, 09:40	30 de jun. de 2026, 09:45
Detalhamento do Contexto	
	
● Usuário 0,4% ● Assistente 39,5% ● Chamadas de Ferramentas 59,9% ● Outros 0,2%	

Análise da refatoração: A refatoração valeu a pena, pois _run_alter_columns tinha 23 branches para um limite de 12, concentrando muitas decisões em um único método. A sessão durou cerca de 5 minutos e consumiu 42.901 tokens, aproximadamente 21% do contexto. O custo foi moderado, mas aceitável, já que o plano foi seguido sem retrabalho e o método ficou mais organizado.

4. Métricas Após a Refatoração

4.1 Radon Após a Refatoração

4.1.1 Complexidade Ciclomática por Arquivo Após a Refatoração

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
piccolo/conf/apps.py	32	3.34	19	107	C	table_finder
piccolo/table.py	75	2.69	18	202	C	_table_str
piccolo/apps/migrations/auto/migration_manager.py	48	3.33	16	160	C	_run_add_columns
piccolo/columns/column_types.py	214	1.75	14	375	C	getattrattribute
piccolo/query/base.py	36	2.39	14	86	C	_load_json_columns

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
piccolo/apps/meta/commands/version.py	1	1.00	1	1	A	version
piccolo/apps/user/piccolo_migrations/2019-11-14T21-52-21.py	1	1.00	1	1	A	forwards
piccolo/apps/user/piccolo_migrations/2020-06-11T21-38-55.py	1	1.00	1	1	A	forwards
piccolo/apps/user/piccolo_migrations/2021-04-30T16-14-15.py	1	1.00	1	1	A	forwards
piccolo/engine/finder.py	1	1.00	1	1	A	engine_finder

4.1.2 Complexidade Ciclomática por Função Após a Refatoração

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
piccolo/conf/apps.py	function	table_finder	C	19
piccolo/table.py	method	_table_str	C	18
piccolo/apps/migrations/auto/migration_manager.py	method	_run_add_columns	C	16
piccolo/apps/migrations/auto/diffable_table.py	method	sub	C	14
piccolo/apps/migrations/commands/backwards.py	method	run_migrations_backwards	C	14

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
piccolo/apps/app/commands/new.py	function	get_app_module	A	1
piccolo/apps/app/commands/new.py	function	new	A	1
piccolo/apps/asgi/commands/new.py	function	print_instruction	A	1
piccolo/apps/fixtures/commands/dump.py	function	dump	A	1
piccolo/apps/fixtures/commands/dump.py	function	dump_to_json_string	A	1

4.1.3 Índice de Manutenibilidade Após a Refatoração

Tabela de piores métricas

arquivo	mi	rank_mi
piccolo/columns/column_types.py	0.0000	C
piccolo/apps/migrations/auto/migration_manager.py	16.3958	B
piccolo/query/methods/select.py	17.0971	B
piccolo/table.py	19.6628	A
piccolo/apps/migrations/auto/serialisation.py	24.0792	A

Tabela de melhores métricas

arquivo	mi	rank_mi
piccolo/init.py	100.0000	A
piccolo/apps/init.py	100.0000	A
piccolo/apps/app/init.py	100.0000	A
piccolo/apps/app/commands/init.py	100.0000	A
piccolo/apps/app/commands/show_all.py	100.0000	A

4.1.4 Métricas Brutas Após a Refatoração

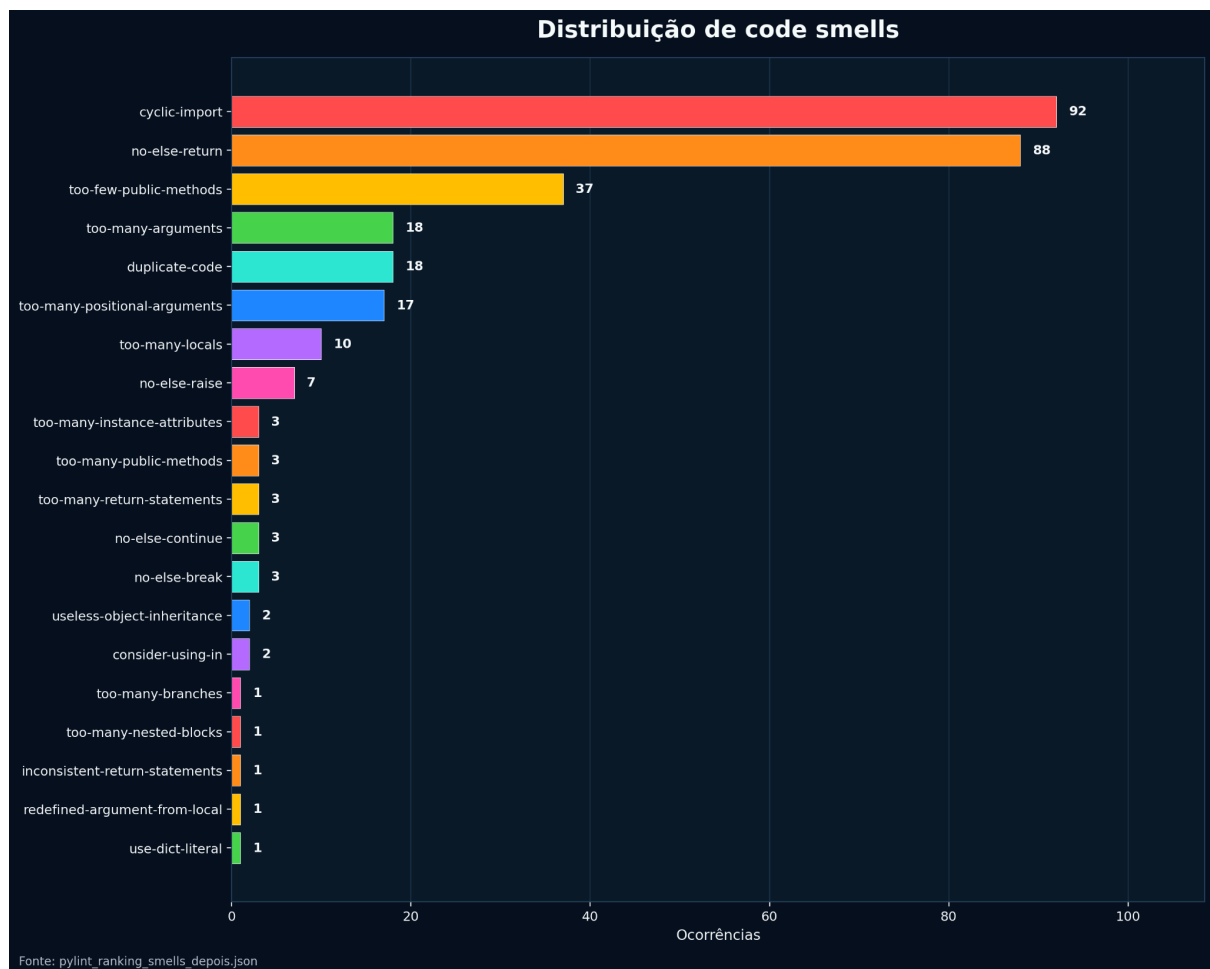
loc total	lloc total	sloc total	comments total	multi
26692	11461	16904	797	3792

4.2 CodeCarbon Após a Refatoração

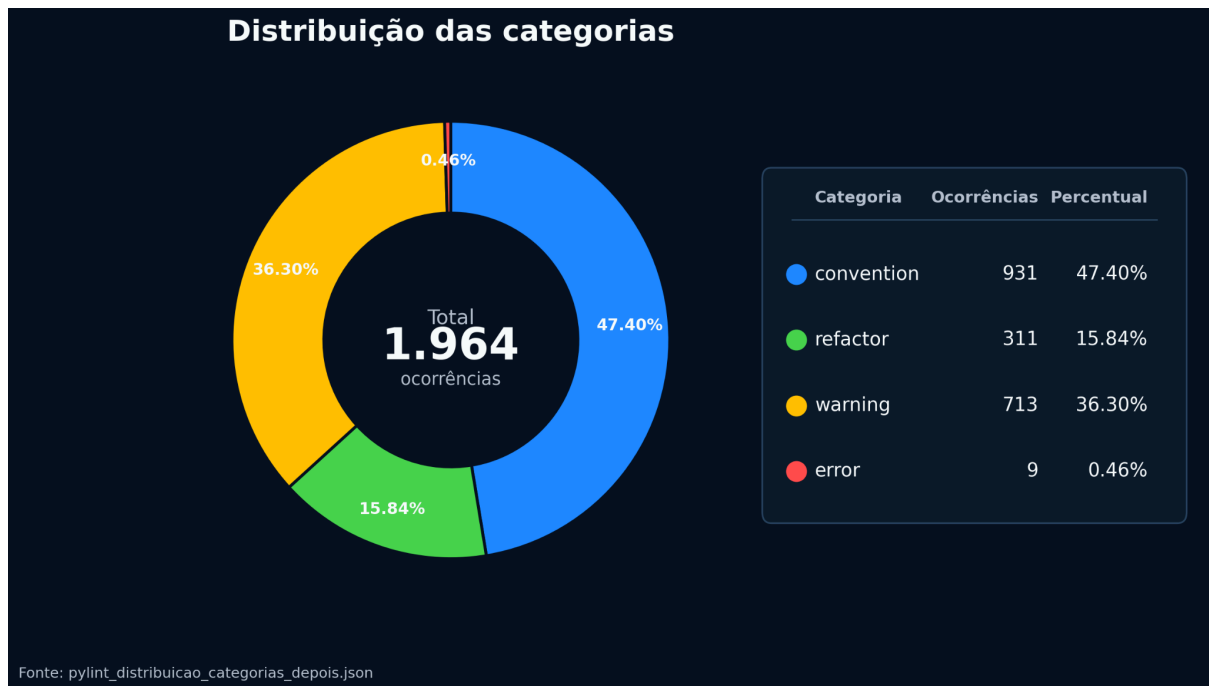
duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
2.850873692999812	1.2774380112103576e-06	4.4808649865725277e-07	8.924042931946322e-07	1.2096553400000528e-05	1.298895769319516e-05

4.3 Pylint Após a Refatoração

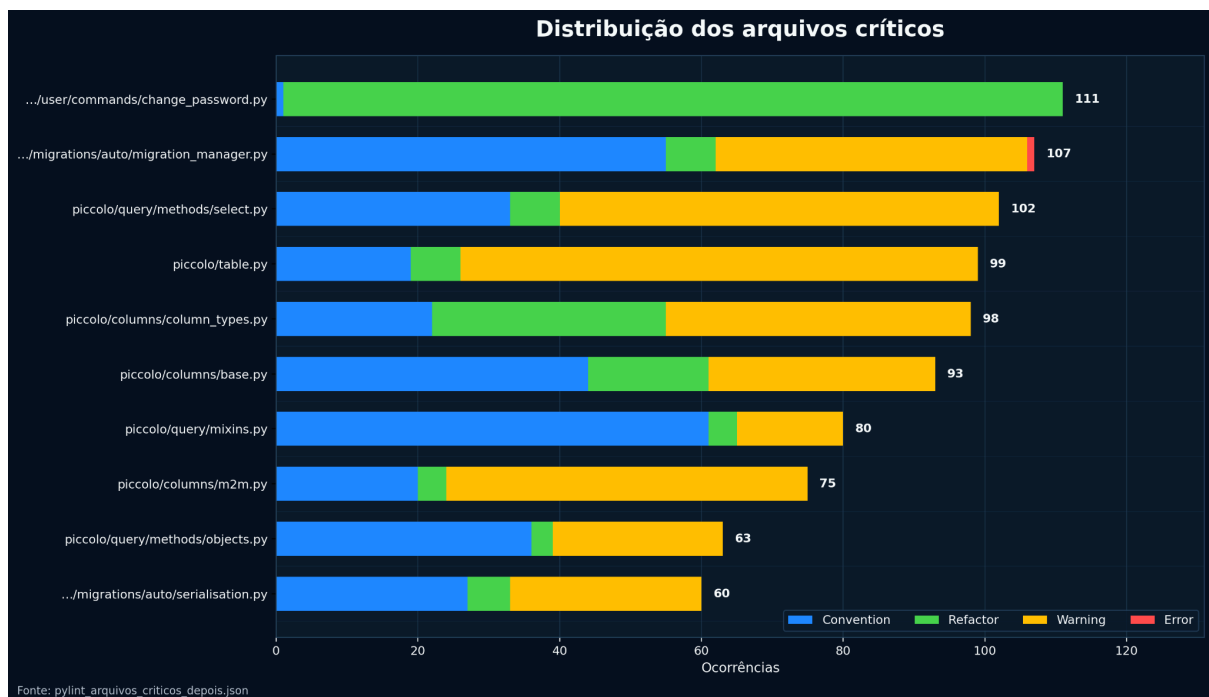
4.3.1 Distribuição do Smells Após a Refatoração



4.3.2 Distribuição dos Problemas de Código Após a Refatoração



4.3.3 Arquivos mais Críticos Após a Refatoração



4.3.4 Score Projeto Após a Refatoração



4.4 Pytest Após a Refatoração

4.4.1 Cobertura de testes Após a Refatoração

File	statements	missing	coverage
piccolo/table.py	555	25	95%
piccolo/columns/base.py	447	50	89%
piccolo/columns/column_types.py	872	110	87%
piccolo/query/methods/select.py	384	38	90%
piccolo/query/methods/alter.py	285	85	70%
piccolo/query/methods/objects.py	265	20	92%
piccolo/query/base.py	219	46	79%
piccolo/query/mixins.py	425	67	84%
piccolo/engine/sqlite.py	328	26	92%
piccolo/engine/postgres.py	255	179	30%
piccolo/engine/base.py	106	9	92%
piccolo/apps/migrations/auto/migration_manager.py	384	182	53%
piccolo/apps/migrations/auto/operations.py	58	0	100%
piccolo/apps/migrations/auto/serialisation.py	368	53	86%
piccolo/apps/migrations/auto/schema_differ.py	266	21	92%
piccolo/apps/migrations/auto/schema_snapshot.py	72	7	90%
piccolo/apps/migrations/auto/diffable_table.py	66	7	89%
piccolo/apps/migrations/commands/forwards.py	79	64	19%
piccolo/apps/migrations/commands/backwards.py	76	62	18%
piccolo/apps/schema/commands/generate.py	397	259	35%
piccolo/utils/pydantic.py	126	5	96%
piccolo/conf/apps.py	254	20	92%

4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração

Quantidade de testes que passaram: 708

Quantidade de testes que falharam: 3

5. Comparação dos Resultados

Após a refatoração, observou-se uma melhora significativa nos code smells escolhidos como foco da atividade. O smell too-many-statements foi eliminado, passando de 6 ocorrências para 0. O

too-many-branches caiu de 13 para 1 ocorrência, e o too-many-instance-attributes foi reduzido de 11 para 3 ocorrências. Assim, a refatoração atingiu seu principal objetivo: reduzir problemas estruturais relacionados a métodos extensos, excesso de ramificações e excesso de atributos em classes.

Em relação à complexidade ciclomática, também houve melhora nos casos mais críticos. Antes da refatoração, a maior complexidade era 40; depois, a maior complexidade passou a ser 19. Além disso, deixaram de existir funções classificadas com ranks D e E, indicando que os trechos mais complexos foram simplificados ou divididos em unidades menores.

Por outro lado, algumas métricas globais apresentaram piora. As métricas brutas mostram que o tamanho do projeto aumentou: o total de linhas passou de 25.642 para 26.692, e o número de linhas lógicas passou de 10.889 para 11.461. Esse crescimento está relacionado principalmente à criação de métodos auxiliares e novas estruturas usadas para distribuir responsabilidades.

O índice de manutenibilidade teve uma leve redução, passando de 82,51 para 82,15 em média. Apesar disso, a mediana permaneceu igual, indicando que a maior parte dos arquivos manteve um bom nível de manutenibilidade. A pequena queda pode ser explicada pelo aumento no tamanho de alguns arquivos após a refatoração.

No Pylint, o resultado também foi misto. O score geral caiu levemente, de 7,91 para 7,89, enquanto a categoria refactor diminuiu de 318 para 311 ocorrências. Isso mostra que os problemas de refatoração foram reduzidos, mas houve aumento em outras categorias, como convention, warning e error.

As métricas de Halstead tiveram pouca variação. O volume aumentou levemente, indicando que o código ficou um pouco maior, mas a dificuldade e o esforço estimado diminuiram um pouco. Isso sugere que, embora a refatoração tenha adicionado novas estruturas, ela também tornou partes do código mais simples de compreender.

As métricas do CodeCarbon indicaram uma pequena redução na duração, no consumo de energia e nas emissões estimadas.

Por fim, os testes com Pytest apresentaram uma pequena piora. Antes da refatoração, 711 testes passaram e nenhum falhou; depois, 708 passaram e 3 falharam. Portanto, embora a maioria dos testes continue passando, esse resultado indica que ainda é necessário investigar as falhas restantes antes de considerar a refatoração totalmente segura.

De modo geral, a refatoração foi positiva para os smells escolhidos e para a redução da complexidade dos trechos mais críticos. Porém, os resultados também mostram efeitos colaterais, como aumento no tamanho do código, leve queda no score do Pylint e surgimento de falhas em alguns testes. Portanto, a qualidade estrutural melhorou nos pontos refatorados, mas ainda há ajustes necessários para estabilizar totalmente o projeto após as mudanças.

6. Percepções sobre a Prática

Experiência geral com a atividade: A atividade proporcionou contato prático com um fluxo real de manutenção de software: desde a configuração do ambiente (venv, dependências, drivers de banco), passando pela coleta de métricas com ferramentas de análise estática, até a refatoração assistida por LLM e a abertura de PRs para um projeto open source ativo. Trabalhar no Piccolo, um ORM com mais de 1900 testes e um codebase maduro, trouxe desafios reais que não apareceriam em exercícios isolados, como dependências de banco de dados, configurações específicas de ambiente e a necessidade de validar cada mudança contra uma suíte de testes extensa antes de avançar.

Code smells: Refatorar too-many-instance-attributes é importante porque classes com excesso de atributos geralmente acumulam responsabilidades demais, dificultando entender o que cada parte do objeto realmente representa e tornando mudanças futuras mais arriscadas. too-many-branches indica lógica condicional complexa que aumenta a chance de bugs e dificulta cobertura de testes completa, já que cada combinação de branches é um caminho de execução distinto. too-many-statements sinaliza funções que fazem coisas demais, violando o princípio de responsabilidade única e tornando o código mais difícil de testar isoladamente e de revisar em PRs. Além dos três tratados, considero duplicate-code (19 ocorrências no projeto) um smell relevante, já que duplicação dificulta manutenção, uma correção precisa ser replicada em múltiplos lugares. cyclic-import (91 ocorrências, o mais frequente do projeto) também seria importante, pois imports circulares indicam acoplamento estrutural problemático entre módulos.

Como benefício, observei que as refatorações tornaram o código mais legível e modular, com responsabilidades bem isoladas em métodos/classes auxiliares, sem alterar a API pública na maioria dos casos. Como não-benefício, algumas refatorações aumentaram o número de arquivos/linhas a manter (mais classes auxiliares, mais properties de compatibilidade), o que pode ser visto como overhead estrutural para smells que tinham baixo limiar de violação.

Uso da LLM na refatoração: A LLM (DeepSeek V4 Flash Free, via OpenCode) entendeu bem os problemas reportados pelo Pylint na maioria dos casos, propondo planos coerentes e tecnicamente corretos. As sugestões foram majoritariamente úteis e aplicáveis, principalmente quando seguiam padrões já validados no próprio codebase. Em alguns casos, o plano inicial era ingênuo, quebrava compatibilidade com chamadas externas que não estavam mapeadas (como no MigrationManager), e exigiu intervenção para questionar e refinar a proposta antes de implementar. Houve também um caso de erro de implementação (métodos movidos para a classe errada), identificado apenas ao rodar a suíte de testes completa.

O uso da LLM acelerou bastante o processo de identificar padrões de refatoração e gerar código boilerplate (properties de compatibilidade, dataclasses), mas exigiu supervisão constante, sem rodar testes a cada etapa e sem questionar criticamente os planos, seria fácil introduzir regressões silenciosas. Projetos open source podem se beneficiar dessa abordagem, desde que haja revisão humana e validação por testes automatizados em cada etapa. Além da refatoração em si, a LLM

ajudou na análise de uso de atributos/funções no codebase, na busca por instâncias que poderiam ser afetadas, e na geração de descrições de PR.

Contribuição com projeto open source: O processo de contribuir envolveu fazer fork do repositório, configurar o ambiente local com as dependências corretas (incluindo drivers de banco que não estavam óbvios à primeira vista), e organizar o trabalho em branches temáticas por tipo de smell. A principal dificuldade foi lidar com configurações de ambiente específicas do projeto (PostgreSQL vs SQLite para testes) e identificar usos não documentados de classes internas que quebravam com mudanças aparentemente seguras. Aprendi a importância de rodar a suíte de testes completa após cada mudança, não confiar apenas no plano da LLM, e a documentar decisões de mudança de plano para rastreabilidade.

O que facilitaria contribuições em projetos open source seria documentação mais explícita sobre como rodar testes localmente (qual engine usar, variáveis de ambiente necessárias) e um guia de contribuição com convenções de commit/PR já estabelecidas, reduzindo a curva de descoberta que tive que percorrer manualmente.

7. Links

- Link do fork: <https://github.com/grazi-lima/piccolo>
- Link do projeto: <https://github.com/piccolo-orm/piccolo>
- Link do PR 1: <https://github.com/piccolo-orm/piccolo/pull/1396>
- Link do PR 2: <https://github.com/piccolo-orm/piccolo/pull/1397>
- Link do PR 3: <https://github.com/piccolo-orm/piccolo/pull/1398>